

운영체제 - 1

남궁명수

[1-1] 가상화: 프로세스의 개념	3
[1] 프로세스의 개념	5
[2] 프로세스 API	7
[3] 프로세스 생성: 좀 더 자세하게	8
[4] 프로세스 상태	10
[5] 자료구조	13
[1-2] 가상화: 프로세스 API	15
[1] fork() 시스템 콜	15
[2] wait() 시스템 콜	17
[3] 드디어, exec() 시스템 콜	18
[4] 왜, 이런 API 를?	20
[5] 여타 API 들	23
[1-3] 가상화: 제한적 직접 실행 원리	24
[1] 기본 원리: 제한적 직접 실행	25
[2] 문제점 1: 제한된 연산	26
[3] 문제점 2: 프로세스 간 전환	29
[4] 문맥(Context) 저장과 복원	30
[5] 병행성과 인터럽트 처리	32
[1-4] 스케줄링: 개요	33
[1] 저수준 기법 정리	34
[2] 워크로드에 대한 가정	36
[3] 스케줄링 평가 항목	36
[4] 선입 선출	37
[5] 최단 작업 우선	38
[6] 최소 잔여시간 우선	40
[7] 새로운 평가 기준: 응답 시간	41
[8] 라운드 로빈(Round Robin Scheduling)	42
[9] 입출력 연산의 고려	48
[1-5] 스케줄링: 멀티 레벨 피드백 큐	50
[1] MLFQ: 기본 규칙	51
[2] 시도 1: 우선순위 변경	52
[3] 시도 2: 우선순위의 상향 조정	54
[4] 시도 3: 더 나은 시간 측정	55
[5] MLFQ 조정과 다른 쟁점들	56

[6] MLFQ: 요약	58
[1-6] 스케줄링: 비례 배분	59
[1] 기본 개념: 추첨권이 당신의 몫을 나타낸다(추첨 스케줄링(Lottery Scheduling))	59
[2] 추첨 기법	61
[3] 구현	63
[4] 추첨 스케줄링의 공정성 분석	65
[5] 추첨권 배분 방식	66
[6] 왜 결정론적(Deterministic) 방법을 사용하지 않는가	67
[7] 요약	69
[1-7] 스케줄링: 멀티프로세서 스케줄링(고급)	70
[1] 배경: 멀티프로세서 구조	72
[2] 동기화를 잊지 마시오	74
[3] 마지막 문제점: 캐시 친화성(Cache Affinity)	77
[4] 단일 큐 스케줄링(SQMS, Single Queue Multiprocessor Scheduling)	78
[5] 멀티 큐 스케줄링(MQMS, Multi Queue Multiprocessor Scheduling)	81
[6] Linux 멀티프로세서 스케줄러	84
[7] 요약	85

[1-1] 가상화: 프로세스의 개념

핵심 질문: CPU가 여러 개 존재한다는 환상을 어떻게 제공하는가?

- 적은 개수의 CPU 밖에 없더라도,
- 운영체제는 어떻게 거의 무한 개에 가까운 CPU가 있는 듯한 환상을 만들 수 있을까?

이 장에서는 운영체제가 제공하는 핵심 개념 중 하나인 프로세스(process)에 대해 논의한다.

일반적으로 프로세스는 실행 중인 프로그램으로 정의한다.

프로그램 자체는 생명이 없는 존재다.

프로그램은 디스크 상에 존재하며 실행을 위한 명령어와 정적 데이터의 묶음이다.

이 명령어와 데이터 묶음을 읽고 실행하여 프로그램에 생명을 불어넣는 것이 운영체제이다.

사용자는 하나 이상의 프로그램을 동시에 실행시키기를 원한다.

예를 들어, 웹 브라우저, 메일 프로그램, 게임, 음악 플레이어 등을 실행하는 데스크톱 또는 랩톱 컴퓨터를 생각해보자.

컴퓨터들은 동시에 수십 혹은 수백 개의 프로세스를 실행하는 것처럼 보인다.

여러 프로그램을 동시에 실행할 수 있으면 시스템을 쉽게 사용할 수 있다.

사용자는 사용 가능한 CPU가 있는지 신경 쓰지 않고 그저 프로그램만 실행시키면 된다.

운영체제는 CPU를 가상화하여 이러한 환상을 만들어낸다.

하나의 프로세스를 실행하고, 얼마 후 중단시키고, 다른 프로세스를 실행하는 작업을 반복하면서 실제 하나 또는 소수의 CPU로 여러 개의 가상 CPU가 존재하는 듯한 환상을 만들어 낸다.

시분할(time sharing)이라 불리는 이 기법은 원하는 수만큼의 프로세스를 동시에 실행할 수 있게 한다.

시분할 기법은 CPU를 공유하기 때문에 각 프로세스의 성능은 낮아진다.

운영체제에서 CPU 가상화를 잘 구현하기 위해 저수준 도구의 고차원적인 “지능”이 필요하다.

저수준 도구를 메커니즘(mechanism)이라 한다.

메커니즘은 필요한 기능을 구현하는 방법이나 규칙을 의미한다.

나중에 문맥 교환(context switch)의 구현에 대해 배우게 될 텐데, CPU에서 프로그램 실행을 잠시 중단하고 다른 프로그램을 실행하는 것을 문맥 교환이라고 한다.

이 시분할 기법은 모든 현대 운영체제들이 채택하고 있다.

정책(Policy)

운영체제의 지능은 정책(policy)의 형태로 표현된다.

정책은 운영체제 내에서 어떤 결정을 내리기 위한 알고리즘이다.

예를 들어, 실행 가능한 여러 프로그램이 있을 때, 운영체제는 어느 프로그램을 실행시켜야 하는가?

- 스케줄링 정책(scheduling policy)이 이러한 결정을 내린다.

이러한 결정을 내리기 위해 스케줄링 정책은 다음을 이용한다.

- 과거 정보(예: 직전 1 분 동안 어떤 프로그램이 자주 실행되었는지)
- 워크로드에 관한 지식(예: 어떤 유형의 프로그램이 실행되었는지)
- 성능 측정 결과(예: 시스템이 대화 성능 혹은 처리량을 높이려 하는지)

팁: 시분할과 공간 분할

시분할은 자원 공유를 운영체제가 사용하는 가장 기본적인 기법 중 하나이다.

한 개체가 잠깐 자원을 사용한 후, 다른 개체가 또 잠깐 자원을 사용하고, 그 다음 개체가 사용하는 방식으로 이 자원(CPU 또는 네트워크 링크 등)을 많은 개체들이 공유한다.

시분할과 자연스럽게 대응되는 개념은 공간 분할(space sharing)이다.

공간 분할은 개체들에게 공간을 분할해 준다.

공간 분할의 예로 디스크가 있다.

디스크는 자연스럽게 공간 분할할 수 있는 자원으로, 블록이 하나의 파일에 할당되면 파일을 삭제하기 전에 다른 파일이 할당될 가능성이 낮다.

[1] 프로세스의 개념

운영체제는 실행 중인 프로그램의 개념을 제공하는데, 이를 프로세스(process)라 한다. 전술한 바와 같이 프로세스는 실행 중인 프로그램이다.

(특정 순간의) 프로세스를 간단하게 표현하려면, 실행되는 동안 접근했거나 영향을 받은 자원의 목록을 작성하면 된다.

프로세스의 구성 요소를 이해하기 위해서는 하드웨어 상태(machine state)를 이해해야 한다. 프로그램이 실행되는 동안 하드웨어 상태를 읽거나 갱신할 수 있다. 이때 가장 중요한 하드웨어 구성 요소는 무엇일까?

프로세스의 하드웨어 상태 중 가장 중요한 구성 요소는 메모리이다. 명령어는 메모리에 저장된다. 실행 프로그램이 읽고 쓰는 데이터 역시 메모리에 저장된다.

프로세스가 접근할 수 있는 메모리(주소 공간(address space)이라 불림)는 프로세스를 구성하는 요소이다.

레지스터도 프로세스의 하드웨어 상태를 구성하는 요소 중 하나이다. 많은 명령어들이 레지스터를 직접 읽거나 갱신한다. 프로세스를 실행하는 데 레지스터도 빠질 수 없다.

- 프로그램 카운터(program counter, PC)
 - 프로그램이 어느 명령어를 실행 중인지 알려준다.
 - 명령어 포인터(instruction pointer, IP)라고도 불린다.
- 스택 포인터(stack pointer), 프레임 포인터(frame pointer)
 - 함수의 변수와 리턴 주소를 저장하는 스택을 관리할 때 사용하는 레지스터이다.

프로그램은 영구 저장장치(persistent storage)에 접근하기도 한다. 이 입출력 정보는 프로세스가 현재 열어 놓은 파일 목록을 포함한다.

팁: 정책과 구현의 분리

많은 운영체제에서 공통된 설계 패러다임은 고수준 정책을 저수준 기법으로부터 분리하는 것이다.

기법은 시스템에 관한 “어떻게”라는 질문에 답을 제공하는 것이라고 생각할 수 있다.

예를 들어, 운영체제는 어떻게 문맥 교환을 하는가? 등이 해당된다.

정책은 “어느 것”이라는 질문에 답한다.

예를 들어, 운영체제는 지금 당장 어느 프로세스를 실행시켜야 하는가? 등이 해당된다.

둘을 분리하면 정책을 변경할 때 기법의 변경을 고민하지 않아도 된다.

따라서 일반적인 소프트웨어 설계 원칙인 모듈성(modularity)의 한 형태이다.

[2] 프로세스 API

실제 프로세스 API에 대한 논의는 다음의 장으로 미루겠지만, 운영체제가 반드시 API로 제공해야 하는 몇몇 기능에 대해 간단히 살펴본다.

이 API들은 형태는 다르지만 모든 현대 운영체제에서 공통적으로 제공된다.

- 생성(create)

운영체제는 새로운 프로세스를 생성할 수 있는 방법을 제공한다.

셸에 명령어를 입력하거나, 응용 프로그램의 아이콘을 더블-클릭하여 프로그램을 실행시키면, 운영체제는 새로운 프로세스를 생성한다.

- 제거(Destroy)

프로세스 생성 인터페이스를 제공하는 것처럼 운영체제는 프로세스를 강제로 제거할 수 있는 인터페이스를 제공한다.

물론, 많은 프로세스는 실행되고 할 일을 다하면 스스로 종료한다.

그러나 프로세스가 스스로 종료하지 않으면 사용자는 그 프로세스를 제거하길 원할 것이고, 필요 없는 프로세스를 중단시키는 API는 매우 유용하다.

- 대기(Wait)

때론 어떤 프로세스의 실행 중지를 기다릴 필요가 있기 때문에 여러 종류의 대기 인터페이스가 제공된다

- 각종 제어(Miscellaneous Control)

프로세스의 제거, 대기 이외에, 여러 가지 제어 기능들이 제공된다.

예를 들어, 대부분의 운영체제는 프로세스를 일시정지하거나 재개(일시정지되었던 프로세스의 실행을 다시 시작)하는 기능을 제공한다.

- 상태(Status)

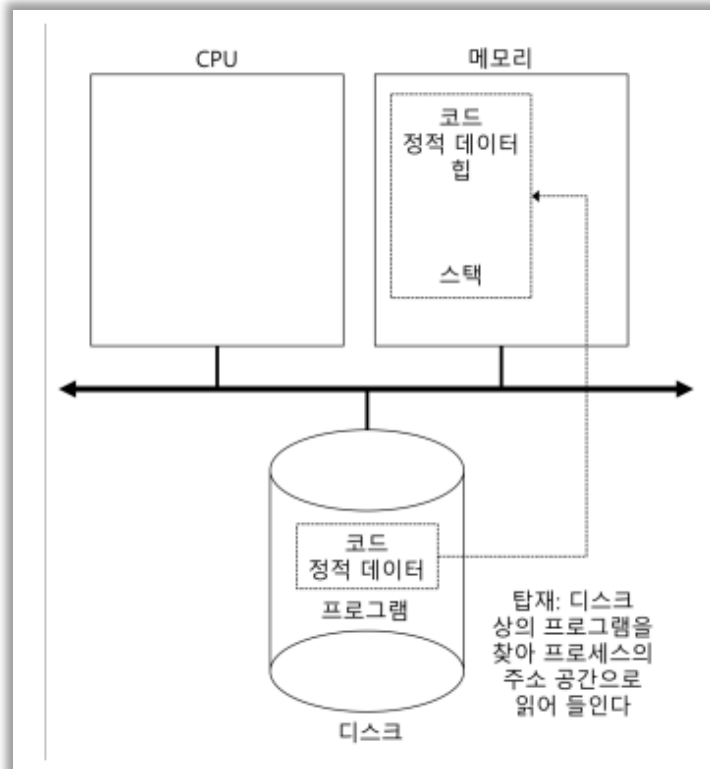
프로세스 상태 정보를 얻어내는 인터페이스도 제공된다.

상태 정보에는 얼마 동안 실행되었는지 또는 프로세스가 어떤 상태에 있는지 등이 포함된다.

[3] 프로세스 생성: 좀 더 자세하게

우리가 밝혀내야 할 한 가지 미스터리는 프로그램이 어떻게 프로세스로 변형되는가이다.

- 운영체제는 어떻게 프로그램을 준비하고 실행시키는가?
- 실제로 어떻게 프로세스를 생성하는가?



프로그램 실행을 위하여 운영체제가 하는 첫 번째 작업은,
프로그램 코드와 정적 데이터(static data, 예: 초기값을 가지는 변수)를 메모리,
즉 프로세스의 주소 공간에 탑재(load)하는 것이다.

프로그램은 디스크 또는 요즘 시스템에서는 플래시 기반 SSD 에 특정 실행 파일 형식으로 존재한다.

코드와 정적 데이터를 메모리에 탑재하기 위해서, 운영체제는 디스크의 해당 바이트를 읽어서 메모리의 어딘가에 저장해야 한다.

초기 운영체제들은 프로그램 실행 전에 코드와 데이터를 모두 메모리에 탑재하였다.
하지만 현대의 운영체제들은 이 작업을 늦추었다.
즉, 프로그램을 실행하면서 코드나 데이터가 필요할 때, 필요한 부분만 메모리에 탑재한다.

코드와 데이터의 늦은 탑재 동작을 정확하게 이해하기 위해서는 페이징(paging)과 스와핑(swapping) 동작에 대한 이해가 필요하다.
이에 대해서는 나중에 메모리 가상화를 논의할 때 자세히 다루게 될 것이다.

지금부터 어떤 프로그램이든 실행시키기 전에, 운영체제는 프로그램의 중요 부분을 디스크에서 메모리로 탑재해야 한다는 것만 기억하자.

코드와 정적 데이터가 메모리로 탑재된 후에도, 프로세스를 실행시키기 전에 운영체제가 해야 할 일이 몇 가지 있다.

일정량의 메모리가 프로그램의 실행시간 스택(run-time stack, 혹은 그냥 스택) 용도로 할당되어야 한다.

이미 알고 있겠지만 C 프로그램은 지역 변수, 함수 인자, 리턴 주소 등을 저장하기 위해 스택을 사용한다.

운영체제는 스택을 주어진 인자로 초기화한다.

특히, main()함수의 인자인 argc 와 argv 벡터를 사용하여 스택을 초기화한다.

운영체제는 프로그램의 힙(heap)을 위한 메모리 영역을 할당한다.

C 프로그램에서 힙은 동적으로 할당된 데이터를 저장하기 위해 사용된다.

프로그램은 malloc()을 호출하여 필요한 공간을 요청하고 free()를 호출하여 사용했던 공간을 반환하여 다른 프로그램이 사용할 수 있도록 한다.

힙은 연결 리스트, 해시 테이블, 트리 등 크기가 가변적인 자료 구조를 위해 사용된다.

프로그램이 실행되면 malloc() 라이브러리 API 를 호출하여 메모리를 요청하고, 운영체제가 이를 충족하도록 메모리를 할당한다.

운영체제는 또 입출력과 관련된 초기화 작업을 수행한다.

예를 들어, Unix 시스템에서 각 프로세스는 기본적으로 표준 입력(STDIN), 표준 출력(STDOUT), 표준 에러(STDERR) 장치에 해당하는 세 개의 파일 디스크립터(file descriptor)를 갖는다.

이 디스크립터들을 사용하여 프로그램은 터미널로부터 입력을 읽고 화면에 출력을 프린트하는 작업을 쉽게 할 수 있다.

입출력, 파일 디스크립터 등에 관해서는 이 책의 영속성 장에서 다룬다.

코드와 정적 데이터를 메모리에 탑재하고, 스택과 힙을 생성하고 초기화하고, 입출력 셋업과 관련된 다른 작업을 마치게 되면, 운영체제는 프로그램 실행을 위한 준비를 마치게 된다.

프로그램의 시작지점, 즉 main()에서부터 프로그램 실행을 시작하는 마지막 작업만 남는다.

main() 루틴으로 분기함으로써(다음 장에서 이를 가능하게 하는 특수 기법을 설명),

운영체제는 CPU 를 새로 생성된 프로세스에게 넘기게 되고 프로그램 실행이 된다.

[4] 프로세스 상태

프로세스가 무엇인지, 어떻게 생성되는지 알게 되었으므로, 프로세스의 상태(state)에 대해 논의한다.

프로세스 상태의 개념은 초기 컴퓨터 시스템에서 등장하였다.

프로세스 상태를 단순화하면 다음 세 상태 중 하나에 존재할 수 있다.

- 실행(Running)

실행 상태에서 프로세스는 프로세서에서 실행 중이다. 즉, 프로세스는 명령어를 실행한다.

- 준비(Ready)

준비 상태에서 프로세스는 실행할 준비가 되어 있지만 운영체제가 다른 프로세스를 실행하고 있는 등의 이유로 대기 중이다.

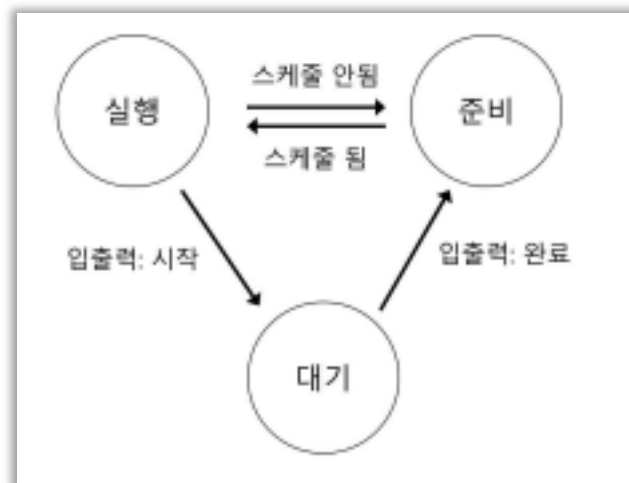
- 대기(Blocked)

프로세스가 다른 사건을 기다리는 동안 프로세스의 수행을 중단시키는 연산이다.

예: 프로세스가 디스크에 대한 입출력 요청을 하였을 때 프로세스는 입출력이 완료될 때까지 대기 상태가 되고, 다른 프로세스가 실행 상태로 될 수 있다.

이러한 상태를 그림으로 표현하면 아래 그림과 같이 될 것이다.

그림에서 보듯이 프로세스는 준비 상태와 실행 상태를 운영체제의 정책에 따라 이동한다. 프로세스는 운영체제의 스케줄링 정책에 따라 스케줄이 되면 준비 상태에서 실행 상태로 전이한다.



실행 상태에서 준비 상태로의 전이는 프로세스가 나중에 다시 스케줄 될 수 있는 상태가 되었다는 것을 의미한다.

프로세스가 입출력 요청 등의 이유로 대기 상태가 되면 요청 완료 등의 이벤트가 발생할 때까지 대기 상태로 유지된다.

이벤트가 발생하면 프로세스는 다시 준비 상태로 전이되고 운영체제의 결정에 따라 바로 다시 실행될 수도 있다.

두 개의 프로세스가 어떻게 전이될 수 있는 지를 보여주는 예를 보자.
 실행 중인 두 프로세스가 있다고 하자.
 각 프로세스는 오직 CPU 만 사용하고 입출력을 행하지 않는다.
 이 경우, 각 프로세스의 상태는 아래 그림과 같은 모양이다.

시간	Process0	Process1	비고
1	실행	준비	
2	실행	준비	
3	실행	준비	
4	실행	준비	Process0 종료
5		실행	
6		실행	
7		실행	
8		실행	Process1 종료

다음 예에서는 첫 번째 프로세스가 어느 정도 실행한 후에 입출력을 요청한다.
 그순간 프로세스는 대기 상태가 되고다른 프로세스에게 실행 기회를 준다.

시간	Process0	Process1	비고
1	실행	준비	
2	실행	준비	
3	실행	준비	Process0이 입출력을 시작한다.
4	대기	실행	Process0은 대기 상태가 되고,
5	대기	실행	Process1이 실행된다.
6	대기	실행	
7	준비	실행	Process0 입출력 종료
8	준비	실행	Process1 종료
9	실행	-	
10	실행	-	Process0 종료

자세히 살펴보면, Process0 은 입출력을 요청하고 요청한 작업이 완료되기를 기다린다.
 프로세스는 디스크를 읽거나 네트워크로부터 패킷을 기다릴 때 대기 상태로 전이한다.
 운영체제는 Process0 이 CPU 를 사용하지 않는다는 것을 감지하고 Process1 을 실행시킨다.
 Process1 이 실행되는 동안 입출력은 완료되고 Process0 은 준비 상태로 다시 전이된다.
 결국 Process1 은 종료되고 Process0 이 실행되어 종료된다.

위와 같은 간단한 예에서조차 운영체제가 내려야 할 결정이 매우 많다는 사실에 주목을 해야 한다.

Process1 을 실행시키기로한 결정은 CPU 를 계속 동작시키므로 자원 이용률을 높인다.

또한, 시스템은 Process0 이 요청한 입출력이 완료되었을 때, Process0 을 바로 실행하지 않고 실행 중이던 Process1 을 계속 실행하였다.

이러한 결정은 좋은 결정이었는지는 확실하지 않다.

당신은 어떻게 생각하는가?

운영체제는 스케줄러를 통해 이러한 결정을 내린다.

운영체제의 스케줄러는 차후에 자세히 다룰 것이다.

여담: 자료구조-프로세스 리스트

운영체제는 다양한 중요 자료 구조들이 많이 존재한다.

프로세스 리스트가 그 중 첫 번째이다.

이 자료 구조는 단순한다.

다수의 프로그램을 동시에 실행할 수 있는 모든 운영체제는 이와 유사한 자료 구조를 갖기고 있고, 이 자료 구조를 이용하여 시스템에서 실행 중인 프로그램을 관리한다.

프로세스의 관리를 위한 정보를 저장하는 자료구조를 프로세스 제어 블록(Process Control Block, PCB)이라 부른다.

각 프로세스에 관한 정보를 저장하는 C 자료 구조를 이야기할 때 부르는 멋진 이름이다.

[5] 자료구조

운영체제도 일종의 프로그램이다.

다른 프로그램들과 같이 다양한 정보를 유지하기 위한 자료 구조를 가지고 있다.

예를 들어, 프로세스 상태를 파악하기 위해 준비 상태의 프로세스들을 위한 프로세스 리스트(process list)와 같은 자료 구조를 유지한다.

또한, 어느 프로세스가 실행 중인지를 파악하기 위한 부가적인 자료 구조도 유지한다.

운영체제는 또 대기 상태인 프로세스도 파악해야 한다.

입출력 요청이 완료되면 운영체제는 적절한 프로세스를 깨워 준비 상태로 다시 전이시킬 수 있어야 한다.

xv6 커널에서 각 프로세스를 추적하기 위해 운영체제가 필요로 하는 정보를 보이고 있다.

Linux, Mac OS X 또는 Windows 같은 운영체제들도 이와 비슷한 프로세스 구조를 가지고 있다.

찾아 보면 훨씬 복잡하다는 것을 알 수 있을 것이다.

운영체제가 관리하고 있는 프로세스 정보를 알 수 있다.

레지스터 문맥(register context) 자료 구조는 프로세스가 중단되었을 때 해당 프로세스의 레지스터값들을 저장한다.

이 레지스터값들을 복원하여(예: 해당 값을 실제 물리 레지스터에 다시 저장함으로써)

운영체제는 프로세스 실행을 재개한다.

문맥 교환(context switch)이라고 알려진 이 기법에 관해서 나중에 더 자세히 다루기로 한다.

```
// 프로세스를 중단하고 나중에 재개하기 위해
```

```
// xv6 가 저장하고 복원하는 레지스터들
```

```
struct context {
```

```
    int eip;
```

```
    int esp;
```

```
    int ebx;
```

```
    int ecx;
```

```
    int edx;
```

```
    int esi;
```

```
    int edi;
```

```
    int ebp;
```

```
};
```

```
// 각 프로세스의 상태
```

```
enum proc_state {
```

```
    UNUSED,
```

```
    EMBRYO,
```

```
    SLEEPING,
```

```
    RUNNABLE,
```

```

    RUNNING,
    ZOMBIE
};

// 프로세스 메모리와 상태를 포함하여
// 각 프로세스에 대해 xv6 가 추적하는 정보
struct proc {
    char *mem;           // 프로세스 메모리 시작 주소
    uint sz;             // 프로세스 메모리 크기
    char *kstack;        // 이 프로세스의 커널 스택의 바닥 주소
    enum proc_state state; // 프로세스 상태
    int pid;             // 프로세스 ID
    struct proc *parent; // 부모 프로세스
    void *chan;          // 0 이 아니면 chan 에서 수면
    int killed;          // 0 이 아니면 종료됨
    struct file *ofile[NOFILE]; // 열린 파일
    struct inode *cwd;    // 현재 작업 디렉토리
    struct context context; // 프로세스를 실행시키려면 여기로 교환
    struct trapframe *tf; // 현재 인터럽트에 해당하는 트랩 프레임
};

```

위 코드에서 실행, 준비, 대기 외에 다른 상태들이 존재하는 것을 볼 수 있다.
초기(initial) 상태를 가지는 시스템도 있다.
프로세스가 생성되는 동안에는 초기 상태에 머무른다.

프로세스가 종료되었지만 메모리에 남아있는 상태인 최종(final) 상태도 있다.(Unix-기반 시스템에서 이 상태는 좀비(zombie) 상태라고 불린다.)

이 상태는 프로세스가 성공적으로 실행했는지를 다른 프로세스(보통은 부모 프로세스가 검사하는 데 유용하다.

이를 위하여 최종 상태를 활용한다.(Unix-기반 시스템에서는 프로세스가 성공적으로 종료되었으면 0 을, 그렇지 않으면 0 이 아닌 값을 반환한다.)

프로세스는 자식 프로세스의 종료를 대기하는 시스템 콜을 호출(예: wait)한다.

이 호출은 종료된 프로세스와 관련된 자원들을 정리할 수 있다고 운영체제에 알리는 역할도 한다.

[1-2] 가상화: 프로세스 API

핵심 질문: 프로세스를 생성하고 제어하는 방법

프로세스를 생성하고 제어하려면 운영체제가 어떤 인터페이스를 제공해야 하는가?

유용하고 편하게 사용하기 위해서 이 인터페이스는 어떻게 설계되어야 하는가?

이번 절에서는 Unix 시스템의 프로세스 생성에 관해 논의한다.

Unix 는 프로세스를 생성하기 위하여 `fork()`와 `exec()` 시스템 콜을 사용한다.

`wait()`는 프로세스가 자신이 생성한 프로세스가 종료되기를 기다리기 원할 때 사용된다.

이제부터 이 인터페이스에 대해 간단한 예제를 이용하여 더 자세하게 설명한다.

[1] `fork()` 시스템 콜

프로세스 생성에 `fork()` 시스템 콜이 사용된다.

미리 경고하자면, 이 시스템콜이 당신이 사용할 시스템 콜 중에서 가장 이해하기 힘든 시스템 콜이다.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int)getpid());

    int rc = fork();

    if(rc < 0) {        // fork 실패 시 종료
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if(rc == 0) {    // 자식 프로세스
        printf("hello, I am child(pid:%d)\n", (int)getpid());
    } else{                // 부모 프로세스는 이 경로를 따라 실행
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int)getpid());
    }

    return 0;
}
```

실행이 시작될 때 프로세스는 “hello world …” 메시지를 출력한다.

이 메시지에는 PID 로도 알려진 프로세스 식별자(process identifier)가 포함된다.

이 프로세스는 29146 이라는 PID 를 가진다.

Unix 시스템에서 PID 는 프로세스의 실행이나 중단과 같이 특정 프로세스를 대상으로 작업을 해야 할 경우 프로세스를 지칭하기 위해 사용된다.

프로세스는 fork() 시스템 콜을 호출한다.

운영체제는 프로세스 생성을 위해 이 시스템 콜을 제공한다.

이상한 부분이 있다: 생성된 프로세스가 호출한 프로세스의 복사본이라는 것이다.

fork() 호출 직후를 살펴보자.

운영체제 입장에서 보면, 프로그램 p1 이 2 개가 존재한다.

두 프로세스가 모두 fork()에서 리턴하기 직전이다.

새로 생성된 프로세스는(일반적으로 자식 프로세스, 생성한 프로세스는 부모 프로세스라 불린다) main() 함수 첫 부분부터 시작하지 않았다는 것을 알 수 있다.(“hello world” 메시지가 한 번만 출력되었다는 것에 유의하기 바란다.)

자식 프로세스는 fork()를 호출하면서부터 시작되었다.

자식 프로세스는 부모 프로세스와 완전히 동일하지는 않다.

자식 프로세스는 자신의 주소 공간, 자신의 레지스터, 자신의 PC 값을 갖는다.

매우 중요한 차이점이 있다.

(매우 중요) fork() 시스템 콜의 반환 값이 서로 다르다.

fork()로 부터 부모 프로세스는 생성된 자식 프로세스의 PID 를 반환하고, 자식 프로세스는 0 을 반환한다.

이 반환 값의 차이로 인해, 부모와 자식 프로세스가 서로 다른 코드를 실행하는 프로그램을 쉽게 작성할 수 있다.

이 프로그램의 출력 결과가 항상 동일하지는 않다.

단일 CPU 시스템에서 이 프로그램을 실행하면, 프로세스가 생성되는 시점에는 2 개(부모와 자식) 프로세스 중 하나가 실행된다.

출력에서 부모 프로세스 실행 후에 자식 프로세스가 실행될 수도 있고, 그 반대 경우도 발생할 수 있다.

CPU 스케줄러는 실행할 프로세스를 선택한다.

스케줄러에 관해서는 곧 자세하게 논의한다.

스케줄러의 동작은 일반적으로 상당히 복잡하고 상황에 따라 다른 선택이 이루어지기 때문에, 어느 프로세스가 먼저 실행된다고 단정하는 것은 매우 어렵다.

이 비결정성으로 인해 멀티 쓰레드 프로그램 실행 시 다양한 문제가 발생한다.

이 책의 병행성 장에서 비결정성에 대해 자세히 다룰 것이다.

[2] wait() 시스템 콜

이제까지 메시지를 출력하는 자식 프로세스를 만들어 보았다.

부모 프로세스가 자식 프로세스의 종료를 대기해야 하는 경우도 발생할 수 있다.

이러한 작업을 위해 wait() 시스템 콜이(혹은 더 많은 기능을 가진 waitpid())있다.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    printf("hello world(pid: %d)\n", (int)getpid());
    int rc = fork();

    if(rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        printf("hello. I am child (pid:%d)\n", (int)getpid());
    } else {
        int wc = wait(NULL);
        printf("hello. I am parent of %d(wc: %d) (pid:%d)\n", rc, wc,
(int)getpid());
    }
    return 0;
}
```

이 예제에서 부모 프로세스는 wait() 시스템 콜을 호출하여 자식 프로세스 종료 시점까지 사니의 실행을 잠시 중지시킨다.

자식 프로세스가 종료되면 wait()는 리턴한다.

wait() 호출을 위와 같이 코드에 추가하면 프로그램은 항상 동일한 결과를 출력한다.

이 코드에서는 항상 자식 프로세스가 먼저 출력을 수행한다는 것을 알 수 있다.

왜 그럴까?

자식 프로세스가 부모 프로세스보다 먼저 실행되면 당연히 자식 프로세스가 먼저 출력된다. 부모 프로세스가 먼저 실행되면 곧바로 wait()를 호출한다. 이 시스템 콜은 자식 프로세스가 종료될 때까지 리턴하지 않는다.

부모가 먼저 실행되더라도 자식 종료 후 wait()가 리턴한다. 그런 후 부모 프로세스가 출력한다.

[3] 드디어, exec() 시스템 콜

프로세스 생성 관련 API 중에서 마지막으로 중요한 시스템 콜은 exec() 시스템 콜이다.

이 시스템 콜은 자기 자신이 아닌 다른 프로그램을 실행해야 할 때 사용한다.

앞선 코드의 fork() 시스템 콜은 자신의 복사본을 생성하여 실행한다.

자신의 복사본이 아닌 다른 프로그램을 실행해야 할 경우에 바로 exec() 시스템 콜이 그 일을 한다.

이 예에서 자식 프로세스는 wc 프로그램을 실행하기 위해 execlp() 시스템 콜을 호출한다.

wc 프로그램은 단어의 개수를 세는 프로그램이다. 사실 예제 프로그램은 자신의 소스 파일인 p3.c 를 인자로 하여 wc 를 실행하고 소스 코드의 행 개수 단어의 개수, 바이트의 개수를 알려준다.

fork() 시스템 콜 만큼 exec() 시스템콜도 이해하기 어렵다.

exec() 시스템 콜은 다음과 같은 과정으로 수행된다.

실행 파일의 이름과(예: wc) 약간의 인자가(예: p3.c)가 주어지면 해당 실행 파일의 코드와 정적 데이터를 읽어 들여 현재 실행 중인 프로세스의 코드 세그먼트와 정적 데이터 부분을 덮어 쓴다.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int)getpid());
    int rc = fork();
    if(rc < 0){
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if(rc == 0){
        printf("hello. I am child (pid: %d)\n", (int)getpid());
        char *myargs[3];

        myargs[0] = strdup("wc");
        myargs[1] = strdup("p3.c");
        myargs[2] = NULL;
```

```

        execvp(myargs[0], myargs); // “wc” 실행
        printf(“this shouldn’t print out”);
    } else{
        int wc = wait(NULL);
        printf(“hello. I am parent of %d (wc:%d) (pid: %d)\n”, rc, wc,
(int)getpid());
    }
    return 0;
}

```

힙과 스택 및 프로그램이 다른 주소 공간들로 새로운 프로그램의 실행을 위해 다시 초기화된다.

그런 다음 운영체제는 프로세스의 argv 와 같은 인자를 전달하여 프로그램을 실행시킨다. 새로운 프로세스를 생성하지는 않는다.

현재 실행 중인 프로그램을(p3) 다른 실행 중인 프로그램으로(wc) 대체하는 것이다. 자식 프로세스가 exec()을 호출한 후에는 p3.c 는 전혀 실행되지 않은 것처럼 보인다. exec() 시스템 콜이 성공하게 되면 p3.c 는 절대로 리턴하지 않는다.

[4] 왜, 이런 API 를?

여기서 이런 의문이 생길 수 있다.

새로운 프로세스를 생성은 간단한 작업 같은데, 왜 이런 이상한 인터페이스를 사용할까?

Unix 의 셸을 구현하기 위해서는 `fork()`와 `exec()`를 분리해야 한다.

그려야만 셸이 `fork()`를 호출하고 `exec()`를 호출하기 전에 코드를 실행할 수 있다.

이때 실행하는 코드에서 프로그램의 환경을 설정하고, 다양한 기능을 준비한다.

셸은 단순한 사용자 프로그램이다.

셸은 프롬프트를 표시하고 사용자가 무언가 입력하기를 기다린다.

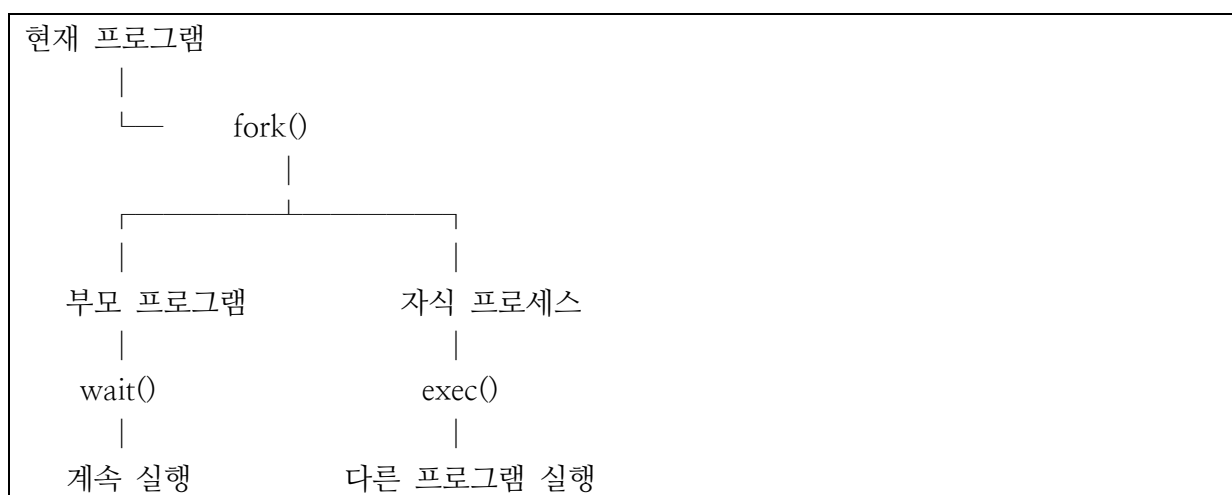
그리고 명령어를 입력한다.(예: 실행 프로그램의 이름과 필요한 인자).

대부분의 경우 셸은 파일 시스템에서 실행 파일의 위치를 찾고 명령어를 실행하기 위하여 `fork()`를 호출하여 새로운 자식 프로세스를 만든다.

그런 후 `exec()`의 변형 중 하나를 호출하여 프로그램을 실행시킨 후 `wait()`를 호출하여 명령어가 끝나기를 기다린다.

자식 프로세스가 종료되면 셸은 `wait()`로부터 리턴하고 다시 프롬프트를 출력하고 다음 명령어를 기다린다.

`fork()`와 `exec()`을 분리함으로써 셸은 많은 유용한 일을 조금 쉽게 할 수 있다.



prompt > wc p3.c > newfile.txt

위의 예제에서 wc 프로그램의 출력은 newfile.txt 라는 출력 파일로 방향이 재지정된다.

(>표시가 재지정을 나타낸다)

이러한 작업을 수행하는 방법은 간단하다.

자식이 생성되고 `exec()`이 호출되기 전에 표준 출력 파일을 닫고, newfile.txt 파일을 연다.

이런 작업을 해 놓으면 곧 실행될 프로그램인 wc 의 출력은 모두 화면이 아니라 파일로 보내진다.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    int rc = fork();

    if (rc < 0) { // fork 실패
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // 자식 프로세스 : 표준 출력을 파일로 재지정

        close(STDOUT_FILENO);

        open("./p4.output", O_CREAT | O_WRONLY | O_TRUNC, S_IRWXU);

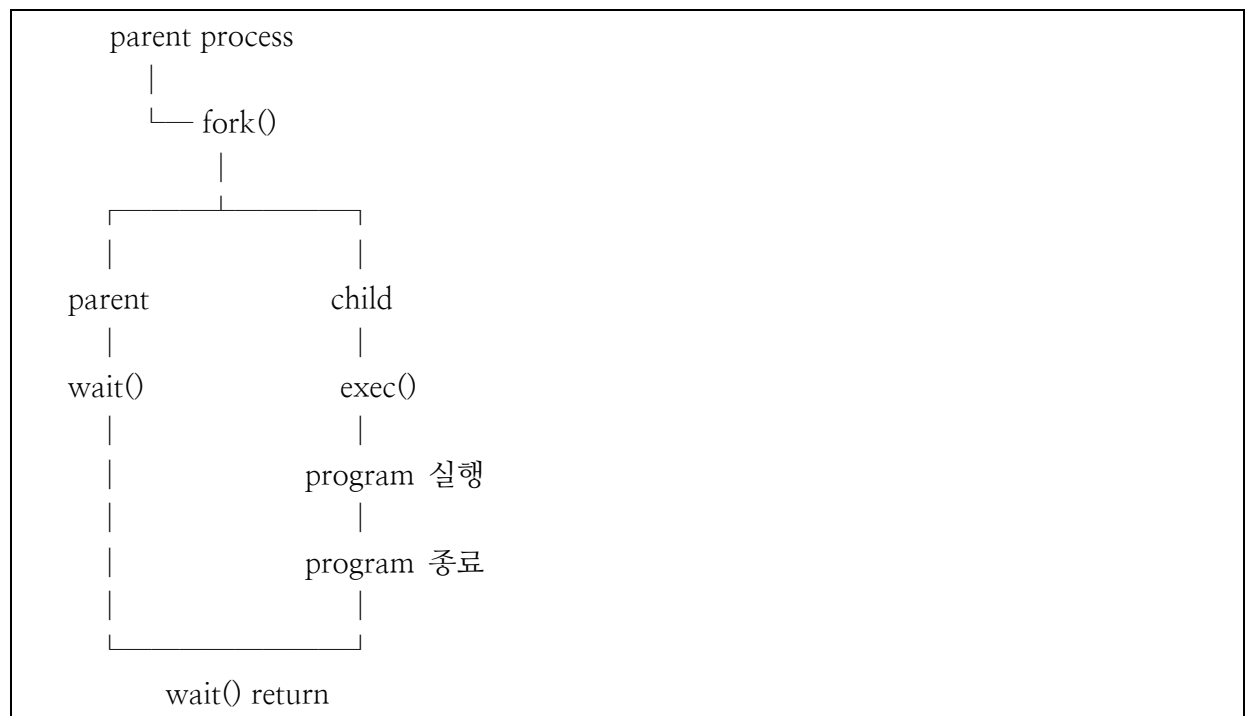
        // 이제 exec 으로 wc 실행

        char *myargs[3];
        myargs[0] = strdup("wc"); // 프로그램 이름
        myargs[1] = strdup("p4.c"); // 인자 (파일 이름)
        myargs[2] = NULL; // 배열 끝 표시

        execvp(myargs[0], myargs); // wc 실행

    } else { // 부모 프로세스
        int wc = wait(NULL);
    }

    return 0;
}
```



[5] 여타 API 들

Unix 시스템에는 `fork()`, `exec()` 및 `wait()` 외에 많은 프로세스 관련 인터페이스가 있다. 예를 들면, `kill()` 시스템 콜은 프로세스에게 시그널을 보내는 데 사용된다. 시그널은 프로세스를 중단시키고(block), 삭제하는 등의 작업에 사용된다. 시그널이라는 운영체제의 메커니즘은 외부 사건을 프로세스에게 전달하는 토대이다. 이 기반 구조는 시그널을 보내거나 전달받는 방법을 모두 포함한다.

유용한 명령어들이 많이 있다.

예를 들면, `ps` 명령어는 어떤 프로세스가 실행 중인지 알아보기 위하여 사용된다.

`ps` 의 유용한 플래그를 알기 위해서는 `man` 페이지를 읽어 보자.

`top` 역시 시스템에 존재하는 프로세스와 그 프로세스가 CPU 및 다른 자원들을 얼마나 사용하고 있는지를 보여주기 때문에 매우 유용하다.

`top` 명령어를 여러번 실행할 경우 자기 스스로가 가장 많은 자원을 사용하고 있다고 지적하는 것이 재밌는 부분이다.

매우 자기중심적인 명령어라고 할 수 있다.

마지막으로 다양한 CPU 측정기가 제공되어 시스템의 부하 정도를 알 수 있다.

예를 들어, Raging Menace software 사의 MenuMeter 를 Macintosh 의 toolbar 에 실행시키면 어느 때건 CPU 의 이용률을 점검할 수 있다.

일반적으로 현재 벌어지고 있는 일에 대해 더 많은 정보를 가질수록 더 유리하다.

[1-3] 가상화: 제한적 직접 실행 원리

핵심 질문: 제어를 유지하면서 효과적으로 CPU 를 가상화하는 방법

운영체제는 효율적인 방식으로 CPU 를 가상화하지만 시스템에 대한 제어를 잃지는 않는다. 제어를 잃지 않기 위해서 하드웨어와 운영체제의 자원이 필수적이다.

종종 운영체제는 작업을 효과적으로 수행하기 위하여 하드웨어가 제공하는 기능을 신중하게 사용한다.

CPU 를 가상화하기 위해서 운영체제는 여러 작업들이 동시에 실행되는 것처럼 보이도록 물리적인 CPU 를 공유한다.

기본적인 아이디어는 간단하다.

한 프로세스를 잠시 동안 실행하고 다른 프로세스를 또 잠깐 실행하고, 이런식으로 계속해서 잠깐씩 실행시키면 된다.

이러한 방식처럼 CPU 를 시간으로 나누어 썸으로써 가상화를 구현할 수 있다.

그러나 이러한 가상화 기법을 구현하기 위해서는 몇 가지 문제를 해결해야 한다.

첫 번째는 성능 저하이다.

시스템에 과중한 오버헤드를 주지 않으면서 가상화를 구현할 수 있을까?

두 번째는 제어 문제이다.

CPU 에 대한 통제를 유지하면서 프로세스를 효율적으로 실행시킬 수 있는 방법이 무엇인가?

운영체제의 입장에서는 자원 관리의 책임자로서 특히 제어 문제가 중요하다. 제어권을 상실하면 한 프로세서가 영원히 실행을 계속할 수 있고 컴퓨터를 장악하거나 접근해서는 안 되는 정보에 접근하게 된다.

제어권을 유지하면서 성능 저하가 없도록 하는 것이 운영체제를 구축하는 데 핵심적인 도전 과제이다.

[1] 기본 원리: 제한적 직접 실행

운영체제 개발자들은 프로그램을 빠르게 실행하기 위하여 제한적 직접 실행(Limited Direct Execution)이라는 기법을 개발하였다.

이 아이디어의 “직접 실행”에 해당하는 부분은 간단하다.

프로그램을 CPU 상에서 그냥 직접 실행시키는 것이다.

따라서 운영체제가 프로그램을 실행하기 시작할 때 프로세스 목록에 해당 프로세스 항목을 만들고 메모리를 할당하며 프로그램 코드를 디스크에서 탑재하고 진입점(예: main() 루틴 혹은 유사한 무엇)을 찾아 그 지점으로 분기하여 사용자 코드를 실행하기 시작한다.

프로세스 목록의 항목을 생성

프로그램 메모리 할당

메모리에 프로그램 탑재

argc/argv 를 위한 스택 셋업

레지스터 내용 삭제

call main() 실행

main() 실행

main 에서 return 명령어 실행

프로세스 메모리 반환

프로세스 목록에서 항목 제거

이 기본적인 직접 실행 프로토콜(아무 제한 없는)을 보이고 있다.

프로그램의 main()으로 분기하고 커널로 되돌아 가기 위해 일반적인 호출과 리턴을 사용하였다.

간단하다. 그러나 이 접근법은 CPU 를 가상화함에 있어 몇 가지 문제를 일으킨다.

첫 번째는 간단한 문제이다.

프로그램을 직접 실행시킨다면 프로그램이 운영체제가 원치않는 일을 하지 않는다는 것을 어떻게 보장할 수 있는가?

두 번째 문제는 프로세스 실행 시, 운영체제는 어떻게 프로그램의 실행을 중단하고 다른 프로세스로 전환시킬 수 있는가.

즉, CPU 를 가상화하는데 필요한 시분할(time sharing) 기법을 어떻게 구현할 수 있는냐는 것이다.

아래에서 이와 같은 질문에 답하면서 CPU 를 가상화하는 데 필요한 사항을 더 잘 이해하게 될 것이다.

이러한 기법을 발전시키는 과정 중에 “제한적”이라는 이름이 어디서 비롯되었는지 보게 될 것이다.

프로그램 실행에 제한을 두지 않으면 운영체제는 어떠한 것도 제어할 수 없으며 따라서 단순한 라이브러리일 뿐이다.

장차 운영체제로 발전하기 위해서는 매우 슬픈 상황이다.

[2] 문제점 1: 제한된 연산

핵심 질문: 제한 연산을 수행하는 방법

프로세스는 입출력 연산을 비롯한 다른 제한된 연산을 수행해야 한다.
그러나, 프로세스는 시스템에 대한 권한이 없기 때문에 제한된 연산을 수행할 수 없다.
이 일을 위해 운영체제와 하드웨어가 할 일은 무엇인가?

팁: 보호된 제어 양도

하드웨어는 두 가지 실행 모드를 제공하여 운영체제를 돕는다.
사용자 모드에서 응용 프로그램은 하드웨어 자원에 대한 접근 권한이 일부 제한되어 있다.

운영체제는 컴퓨터의 모든 자원에 대한 접근 권한을 커널 모드에서 가진다.
이를 위하여 커널 모드로 진입하기 위한 trap 명령어와 사용자 모드로 돌아가기 위한 return-from-trap 명령어가 제공된다.
또한, 운영체제가 하드웨어에게 트랩 테이블(trap table)의 메모리 주소를 알려주기 위한 명령어도 함께 존재한다.

프로그램을 CPU에서 직접 실행하는 장점은 빠르게 실행된다는 것이다.
그러나 이렇게 직접 실행하면 새로운 문제가 생긴다.

예를 들어, 프로세스가 특수한 연산을 수행하려고 한다고 가정한다.
여기에는 디스크 입출력 요청, CPU 또는 메모리와 같은 시스템 자원 할당 요청 등이 포함된다.
프로세스에게 모든 권한을 그대로 주면, 원하는 대로 하드웨어가 제어할 수 있다.
하지만 이는 안전한 시스템을 구축하는 데 방해가 된다.

예를 들어, 파일 시스템 접근을 허용하기 전에 권한 검사를 해야 하는 경우를 생각하자.
프로세스가 디스크에 마음대로 접근할 수 있다면, 권한 검사의 의미가 없어진다.

사용자 모드와 커널모드

이 때문에 사용자 모드가 도입되었다.
사용자 모드에서 실행되는 코드는 할 수 있는 일이 제한된다.
예를 들어, 디스크 입출력 같은 특권 명령을 직접 실행할 수 없다.
만약 실행하려면 프로세스는 예외를 발생시키고, 운영체제는 해당 프로세스를 종료할 수 있다.

반대로 커널 모드에서는 운영체제의 중요한 코드가 실행되며, 모든 명령어와 시스템 자원을 사용할 수 있다.

사용자 프로세스가 특권 명령을 실행해야 할 때

사용자 프로세스가 디스크를 읽거나, 메모리를 할당하는 등의 특권 명령을 수행해야 할 때는 시스템 콜(system call)을 이용한다.

- 거의 모든 현대 하드웨어는 시스템 콜을 제공한다.
- 이를 통해 파일 시스템 접근, 프로세서 생성과 제거, 프로세스 간 통신, 메모리 할당 등 운영체제의 핵심 기능을 안전하게 사용할 수 있다.
- 초기 Unix 시스템은 약 20 개의 시스템 콜을 제공했지만, 현대 운영체제는 수백 개를 제공한다.

시스템 콜 실행 과정

1. 프로세스는 trap 특수 명령어를 실행한다.
 - 이 명령어는 커널 모드로 전환하면서 운영체제 코드로 분기한다.
2. 커널 모드에서 운영체제는 요청된 작업을 수행한다.
3. 완료되면 return-from-trap 명령어를 사용하여 다시 사용자 모드로 돌아가 프로세스 실행을 재개한다.

하드웨어는 이 과정에서 레지스터와 프로그램 카운터를 커널 스택에 저장한다.

트랩 핸들러와 안전성

운영체제는 트랩이 발생했을 때 어떤 코드를 실행할지 엄격하게 통제한다.

- 프로세스가 커널 내부의 임의 코드로 점프할 수 없도록 제한한다.
- 커널은 부팅 시 트랩 테이블(trap table)을 생성하여 하드웨어에 알려준다.
- 하드웨어는 이 정보를 기억하고, 인터럽트나 시스템 콜 발생 시 지정된 트랩 핸들러로 분기한다.

실행 흐름

1. 커널 부팅 시 트랩 테이블 초기화
2. CPU 가 사용자 프로세스를 실행 → 커널 스택 및 레지스터 준비
3. 시스템 콜 발생 → trap → 커널 모드
4. 운영체제가 요청 처리 → return-from-trap → 사용자 모드로 복귀
5. 프로세스가 main() 수행 후 종료 → exit() 호출 → 다시 trap → 커널 정리 작업

trap

trap 은 CPU 가 제공하는 특수한 명령어 또는 예외 처리 메커니즘을 의미한다.

- 사용자가 직접 실행할 프로그램(사용자 모드)이 특권 명령어(디스크 I/O, 메모리 관리 등)를 수행하려고 하면 바로 실행할 수 없으므로 CPU 가 예외를 발생시킨다.
- 이 예외를 운영체제가 처리할 수 있도록 커널 모드로 전환하는 것이 trap 이다.

여담: 왜 시스템 콜은 프로시저 콜과 비슷하게 보일까

C 프로그램에서 `open()`이나 `read()`같은 시스템 콜을 호출하는 모습은 일반 함수 호출과 거의 똑같아 보인다.

하지만 내부적으로는 운영체제가 이 호출을 특권 작업으로 구분하고 처리한다.

핵심 아이디어

- 시스템 콜은 진짜 프로시저 호출이지만, 내부에 `trap` 명령어가 숨어 있다.
- 개발자는 일반 함수처럼 호출하지만, 실제로는 `trap` 을 통해 커널로 진입한다.

작동 과정

1. C 코드에서 `open()`을 호출하면, C 라이브러리(Libc)의 함수가 먼저 실행된다.
2. 라이브러리 함수는 커널과 약속된 규약(인자 위치, 시스템 콜 번호 등)에 따라 필요한 정보를 레지스터 또는 스택에 저장한다.
3. 그런 다음 `trap` 명령어를 실행하여 CPU 를 커널 모드로 전환한다.
4. 커널은 시스템 콜을 수행하고, 완료되면 `return-from-trap` 을 통해 사용자 모드로 돌아간다.
5. 라이브러리는 반환 값을 받아 호출한 코드에 전달한다.

의미

- 개발자는 어셈블리 코드를 직접 작성할 필요가 없다.
- 시스템 콜이 일반 함수처럼 보이지만, 안전하게 커널 기능을 실행할 수 있는 장치가 숨겨져 있는 구조이다.

[3] 문제점 2: 프로세스 간 전환

핵심 질문: CPU 를 어떻게 다시 획득할 수 있는가

운영체제는 어떻게 CPU 를 다시 획득하여 프로세스를 전환할 수 있는가?

직접 실행의 두 번째 문제점은 프로세스 간 전환(context switch)이 필요하다는 점이다.

- 운영체제는 현재 실행 중인 프로세스를 계속 실행할지, 아니면 멈추고 다른 프로세스를 실행할지 결정해야 한다.

- 그러나 실제로 CPU 에서 프로세스가 실행 중이라는 것은 운영체제가 해당 프로세스를 직접 제어하지 못하고 있다는 것을 의미한다.

- 즉, CPU 가 프로세스를 실행 중일 때 운영체제는 직접 개입할 수 없다.

이 문제를 해결하기 위해 다양한 방식이 고안되었다.

[3-1] 협조적(cooperative) 방식

과거 일부 시스템에서는 협조적 방식을 사용했다.

- 초기 Macintosh OS, Xerox Alto 시스템 등이 대표적이다.

- 핵심 아이디어: 운영체제는 프로세스가 합리적으로 행동하여 CPU 를 포기할 것이라고 신뢰한다.

CPU 포기 방식

- 프로세스는 주기적으로 시스템 콜을 호출하여 운영체제에 제어권을 반환한다.

- 예시: 파일 열기, 읽기, 메시지 전송, 새 프로세스 생성 등

- 일부 시스템은 `yield()` 같은 시스템 콜을 제공하여, 프로세스가 명시적으로 CPU 를 양보할 수 있도록 한다.

문제점

- 만약 프로세스가 버그나 악의적 루프에 빠지면, 운영체제는 제어권을 되찾지 못한다.

- 이 경우, 과거에는 시스템 재부팅과 같은 극단적인 방법을 사용할 수 밖에 없었다.

[3-2] 비협조적(preemptive) 방식

현대 운영체제는 운영체제가 전권을 행사하는 비협조적 방식을 사용한다.

- 하드웨어의 타이머 인터럽트를 이용한다.

- 타이머 장치는 일정 시간(예: 수 밀리초)마다 인터럽트를 발생시킨다.

- 인터럽트 발생 시, 현재 실행 중인 프로세스가 강제로 중단되고, 미리 구성된 인터럽트 핸들러가 실행된다.

운영체제 관점에서의 장점

- CPU 제어권을 강제로 확보할 수 있다.

- 현재 프로세스를 중단하고 다른 프로세스를 실행할 수 있다.

[3-3] 인터럽트와 시스템 콜의 유사성

- 타이머 인터럽트 발생 시, 하드웨어는 현재 프로세스 상태를 저장한다.
- 이후 `return-from-trap` 명령어를 통해 프로세스를 다시 실행할 수 있다.
- 이 과정은 시스템 콜 호출 시 `trap` 명령어와 매우 유사하다.
 - 레지스터 저장, 커널 모드 전환, 작업 수행, 사용자 모드 복귀 등

즉, 협조적 방식과 비협조적 방식 모두 시스템 콜과 `trap` 개념들을 활용하며, 현대 운영체제는 인터럽트를 이용한 비협조적 방식을 표준으로 사용한다.

[4] 문맥(Context) 저장과 복원

운영체제가 프로세스 제어권을 다시 획득하면, 현재 실행 중인 프로세스를 계속 실행할지, 다른 프로세스로 전환할지를 결정해야 한다.

- 이 결정은 스케줄러가 수행하며, 스케줄러 정책에 따라 어떤 프로세스를 실행할지 선택한다.

프로세스 전환과 문맥 교환

다른 프로세스로 전환하기로 결정되면, 운영체제는 문맥 교환(context switch)을 수행한다.

- 문맥 교환 핵심 작업:
 1. 현재 실행 중인 프로세스의 레지스터 값을 커널 스택 또는 프로세스 구조체에 저장
 2. 곧 실행될 프로세스의 레지스터 값을 커널 스택에서 복원
- 이렇게 해야 `return-from-trap` 명령어가 실행될 때 이전 프로세스가 아닌 새로운 프로세스로 실행이 이어진다.

문맥 교환의 세부 과정

1. 저수준 레지스터 저장
 - 범용 레지스터, 프로그램 카운터(PC), 커널 스택 포인터 등 저장
2. 실행할 프로세스 준비
 - 새로운 프로세스의 커널 스택과 레지스터를 복원
 - 스택 포인터를 새로운 커널 스택으로 전환
3. 프로세스 실행
 - `return-from-trap` 명령어로 새로운 프로세스 문맥에서 실행 시작

두 번의 레지스터 저장/복원

문맥 교환 동안 레지스터 저장/복원이 두 번 일어난다.

1. 타이머 인터럽트 발생 시
 - 하드웨어가 현재 실행 중인 프로세스의 사용자 레지스터를 암묵적으로 저장
 - 저장 장소: 해당 프로세스의 커널 스택
2. 운영체제가 프로세스 전환 결정 시
 - 커널 레지스터를 현재 프로세스 구조체에 저장
 - 새로운 프로세스 구조체에서 레지스터 복원

이렇게 하면 운영체제는 A 에서 B 로 전환할 때, B 프로세스를 현재 커널 문맥에서 실행하는 것처럼 만들 수 있다.

예: 타이머 인터럽트를 통한 프로세스 전환

1. 부팅 시
 - 커널은 트랩 테이블 초기화
 - syscall 핸들러, 타이머 핸들러 주소를 저장
2. 실행 중인 프로세스 A
 - 일정 시간이 지나면 타이머 인터럽트 발생
 - 하드웨어가 A 의 레지스터를 커널 스택에 저장
 - 커널 모드로 전환, 트랩 핸들러 실행
3. 스케줄러가 B 로 전환 결정
 - switch() 호출:
 - A 의 레지스터를 구조체에 저장
 - B 의 레지스터 복원
 - B 의 커널 스택으로 스택 포인터 전환
 - return-from-trap 실행 -> B 프로세스 실행

[5] 병행성과 인터럽트 처리

문제 상황

- 시스템 콜을 처리하는 도중 타이머 인터럽트가 발생하면 어떻게 될까?
- 이미 한 인터럽트를 처리하고 있을 때 다른 인터럽트가 발생하면 운영체제는 어떻게 처리해야 할까?

이러한 문제는 병행성(concurrency)과 관련되어 있으며, 운영체제가 신중하게 설계해야 하는 부분이다.

기본 해결책: 인터럽트 불능화

- 운영체제는 인터럽트 처리 도중 다른 인터럽트를 일시적으로 막는 방법을 사용한다.
 - 즉, 하나의 인터럽트가 처리되는 동안 다른 인터럽트는 CPU 에 전달되지 않는다.
- 주의점:
 - 인터럽트를 너무 오래 불능화하면, 중요한 인터럽트를 놓치게 된다.
 - 기술적으로도 바람직하지 않다.

자료 구조 보호: 락(lock)기법

- 커널 내부 자료 구조에 동시에 접근을 방지하기 위해 락을 사용
- 락을 통해 커널은 여러 활동을 동시에 진행할 수 있다.
- 하지만
 - 락은 설계가 복잡할 수 있다.
 - 잘못 설계되면 버그나 교착 상태(deadlock)를 유발한다.

[1-4] 스케줄링: 개요

핵심 질문: 스케줄링 정책은 어떻게 개발하는가

- 스케줄링 정책을 생각하기 위한 기본적인 프레임워크를 어떻게 만들어야 하는가?
- 핵심 가정은 무엇인가?
- 어떤 평가 기준이 중요한가?
- 컴퓨터 시스템의 초창기에 사용되었던 기본 접근법은 무엇인가?

이제 프로세스를 실행하기 위한 문맥 교환 등의 저수준 기법에 대해서는 분명하게 이해하고 있어야 한다.

그렇지 않다면 한 두 장 이전으로 돌아가서 이러한 기법이 어떻게 동작하는지에 대한 설명을 다시 읽기 바란다.

운영체제 스케줄러의 고수준 정책에 관해서는 이해가 필요하다.

이제부터 다양한 정책 스케줄링 정책을 소개하고 그에 관한 이해를 높일 것이다.

이러한 정책은 원칙이라고도 불리며 많은 똑똑하고 부지런한 사람들이 오랫동안 개발한 정책들이다.

사실 스케줄링의 기원은 컴퓨터 시스템 개발 이전으로 거슬러 올라간다.

초기의 방법들은 생산 관리 분야에서 개발되었으며 컴퓨터에 적용되었다.

이 사실은 그렇게 놀라운 사실이 아니다.

생산 공정과 기타 많은 인간의 행동 역시 스케줄링이 필요하고, 효율성에 대한 요구 등, 공통된 해결 사항이 존재하기 때문이다.

[1] 저수준 기법 정리

[1-1] 사용자 모드와 커널 모드

운영체제는 시스템을 위해 두 가지 실행 모드를 사용한다.

- User Mode

- 일반 사용자 프로그램이 실행되는 모드
- 특권 명령어 실행 불가능
- 디스크 접근, 메모리 관리 등의 작업 불가

- Kernel Mode

- 운영체제가 실행되는 모드
- 모든 하드웨어 접근 가능
- 시스템 자원 관리 수행

사용자 프로그램이 특권 작업을 수행하려면 커널 모드로 전환해야 한다.

[1-2] 시스템 콜과 Trap

사용자 프로그램이 운영체제 기능을 요청할 때 시스템 콜(system call)을 사용한다.

시스템 콜의 동작 과정:

1. 사용자 프로그램이 open(), read() 같은 시스템 콜 호출
2. C 라이브러리가 trap 명령어 실행
3. CPU가 커널 모드로 전환
4. 커널이 요청된 작업 수행
5. 작업 완료 후 return-from-trap 실행
6. 다시 사용자 모드로 복귀

[1-3] 타이머 인터럽트

CPU를 직접 실행하는 구조에서는 프로세스가 CPU를 독점할 위험이 있다.

이를 해결하기 위해 타이머 인터럽트를 사용한다.

동작 방식:

1. 운영체제가 타이머 설정
2. 일정 시간이 지나면 인터럽트 발생
3. CPU는 현재 프로세스를 중단
4. 운영체제 인터럽트 핸들러 실행
5. 운영체제가 CPU 제어권 획득

이를 통해 운영체제는 프로세스를 강제로 중단할 수 있다.

[1-4] 문맥 교환

운영체제가 다른 프로세스를 실행하기로 결정하면 문맥 교환이 발생한다.

문맥 교환 과정:

1. 현재 프로세스의 상태 저장
 - 레지스터
 - 프로그램 카운터
 - 스택 포인터
2. 해당 정보를 프로세스 구조체(PCB)에 저장
3. 다음 실행할 프로세스의 상태 복원
4. return-from-trap 실행
5. 새로운 프로세스를 시행

[2] 워크로드에 대한 가정

가장 먼저 프로세스에 대하여 몇 가지 가정을 할 것이다.

일련의 프로세스를 실행하는 상황을 워크로드(workload)라고 부르기도 한다.

워크로드를 결정하는 것은 정책 개발에 매우 중요한 부분이다.

워크로드에 관해서 더 잘 알수록 그에 맞는 정책을 정교하게 손질할 수 있다.

우리가 지금 설명에서 사용할 워크로드에 대한 가정은 비현실적이다.

하지만, 당장 논의하는 데 아무 문제 없다.

앞으로 논의가 진행됨에 따라 가정을 완화시킬 것이고 최종적으로 제대로 동작하는 스케줄링 정책을 만들게 될 것이다.

우리가 시스템에서 실행 중인 프로세스 혹은 작업(job)에 대해 다음과 같은 가정을 한다.

1. 모든 작업은 같은 시간 동안 실행된다.
2. 모든 작업은 동시에 도착한다.
3. 각 작업은 시작되면 완료될 때까지 실행된다.
4. 모든 작업은 CPU 만 사용한다.(즉, 입출력을 수행하지 않는다.)
5. 각 작업의 실행 시간은 사전에 알려져 있다.

이러한 가정들이 비현실적이라고 이미 언급하였다.

특히, 각 작업의 실행 시간이 미리 알려져 있다는 가정은 매우 비현실적이다.

[3] 스케줄링 평가 항목

워크로드에 대한 가정 이외에 스케줄링 정책의 비교를 위해 스케줄링 평가 항목을 결정해야 한다. 스케줄링의 경우 다양한 평가 기준이 존재한다.

우선, 문제를 간단하게 하기 위해 반환 시간(turnaround time)이라는 하나의 평가 기준을 사용한다.

작업 반환 시간은 작업이 완료된 시각에서 작업이 시스템에 도착한 시각을 뺀 시간으로 정의된다.

$$T_{\text{반환 시간}} = T_{\text{완료 시간}} - T_{\text{도착 시간}}$$

모든 작업은 동시에 도착한다고 가정했으므로 $T_{\text{도착 시간}} = 0$ 으로 생각할 수 있다.

$T_{\text{반환 시간}} = T_{\text{완료 시간}}$ 이다. 이 가정은 완화해 나갈 것이다.

반환 시간은 성능 측면에서의 평가 기준이라는 것을 주목해야 한다.

다른 평가 기준으로 공정성(fairness)이다.

예를 들면, Jain's Fairness Index 에 따라 측정된다.

성능과 공정성은 스케줄링에서는 서로 상충되는 목표이다.

예를 들어, 스케줄러는 성능을 극대화하기 위해 몇몇 작업의 실행을 중지하며, 결과적으로 공정성이 악화된다.

[4] 선입 선출

가장 기초적인 알고리즘은 선입 선출(First In First out, FIFO) 또는 선도착선처리(First Come First Served, FCFS) 스케줄링이라고 알려져 있다.

FIFO는 많은 장점을 가진다.

단순하고 구현하기 쉽다. 우리의 가정 하에서 매우 잘 동작한다.

간단한 예를 스케줄 해보자.

시스템에 3개의 작업 A, B, C가 거의 동시에 도착했다고 가정한다. ($T_{\text{도착 시간}} = 0$).

간발의 차이로 A, B, C 순서대로 도착했다고 가정하자.

또한, 각 작업은 10초 동안 실행된다고 가정하자.

이 작업들의 평균 반환 시간은 얼마인가?

A는 10, B는 20, C는 30에 종료했다는 것을 알 수 있다.

세 작업의 평균 반환 시간은 $\frac{10+20+30}{3} = 20$ 이다. 반환 시간의 계산은 쉽다.

이제 가정 중 한 가지를 완화하자.

구체적으로 1번 가정을 완화하여 작업을 실행 시간이 모두 같지 않다고 가정하자.

FIFO는 어떻게 동작하는가? FIFO는 어떤 워크로드에서 좋지 않은 성능을 보일까?

FIFO 스케줄링이 어떤 문제를 야기하는지 같이 생각해본다.

구체적으로 다시 A, B, C 세 개의 작업을 가정한다.

그러나 이번에는 A는 100초, B와 C는 10초 동안 실행된다.

작업 A가 B와 C보다 먼저 100초 동안 실행된다.

따라서 시스템의 평균 반환 시간은 110초로 늘어난다. $\frac{100+110+120}{3} = 110$

이 문제점은 convoy effect라고 칭해지며 짧은 시간 동안 자원을 사용할 프로세스들이 자원을 오랫동안 사용하는 프로세스의 종료를 기다리는 현상을 말한다.

이 스케줄링 시나리오는 슈퍼마켓에서 줄 서서 계산을 기다릴 때, 앞 사람이 카트 세 개에 물건을 가득 싣고 있는 경우와 느낌이 비슷하다. (한참 걸리겠지?)

그러면 어떻게 처리해야 할까?

작업 실행 시간이 다른 경우 더 좋은 알고리즘은 무엇인가?

팁: SJF의 원칙

최단 작업 우선 원칙은 평균 반환 시간이 강조되는 모든 시스템에 적용될 수 있는 일반적인 스케줄링 원리이다.

당신이 기다린 경험이 있는 모든 줄을 생각해 보라.

문제의 장소가 고객 만족을 중요시한다면 SJF의 사용을 고려했을 가능성이 높다.

예를 들어, 식료품 가게들은 적은 개수의 물건을 구매한 사람들이 다가올 겨울을 준비하는 가족 뒤에 서서 마냥 기다리지 않도록 “소량 구매 계산대”를 갖추고 있다.

[5] 최단 작업 우선

이 문제를 간단히 해결할 수 있다.

사실 이 아이디어는 오퍼레이션 리서치 분야에서 차용하여 컴퓨터 시스템의 작업 스케줄링에 적용한 것이다.

이 새로운 스케줄링 개념을 최단 작업 우선(Shortest Job First, SJF)으로 이름 자체가 정책을 설명하고 있기 때문에 기억하기 쉽다.

이 원칙은 가장 짧은 실행 시간을 가진 작업을 먼저 실행시킨다.

위의 예를 이번에는 SJF 로 스케줄 해보자.

A의 실행 시간 100, B와 C의 실행 시간 10, 도착 시간은 아슬아슬하게 A, B, C

SJF가 평균 반환 시간 기준으로 더 나은 성능을 보이는지 보인다.

B, C, A 순서로 실행시킴으로써 SJF는 평균 반환 시간을 110 초에서 50 초($\frac{10+20+120}{3} = 50$)로 2 배 이상 향상시켰다.

모든 작업이 동시에 도착한다면 SJF가 최적(optimal)의 스케줄링 알고리즘임을 증명할 수 있다.

지금은 이론이나 오퍼레이션 리서치 수업이 아니라 시스템 수업을 듣고 있기 때문에 이에 대한 증명은 하지 않을 것이다.

SJF라는 좋은 스케줄링 기법을 개발하였지만 가정이 아직 비현실적이다.

이제 가정을 완화해 보자.

특히, 가정 2를 완화하여 이제 모든 작업은 동시에 도착하는 것이 아니라 임의의 시간에 도착할 수 있다고 가정한다.

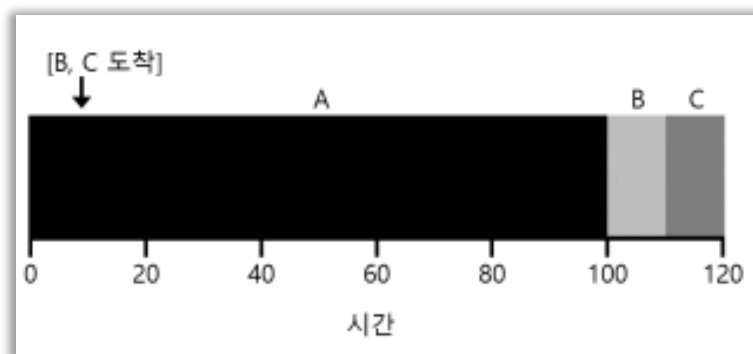
이러면 어떤 문제가 발생하는가?

자 문제가 무엇인지 설명하기 위하여 예를 들어보자.

이번에는 A는 $t = 0$ 에 도착하고 100초의 실행 시간을 가지고 있다.

반면에 B와 C는 $t = 10$ 에 도착하고 각각 10초의 실행 시간을 가진다고 가정하자.

순수한 SJF로 스케줄 했을 경우의 스케줄이 그림에 나와 있다.



그림에서 알 수 있듯이 B와 C가 A 바로 뒤에 도착한다고 가정하더라도 A가 끝날 때까지 기다릴 수 밖에 없어서 이전의 convoy 문제가 다시 발생한다.

이 경우 평균 반환 시간은 $103.33(\frac{100+(110-10)+(120-10)}{3} = 103.33)$ 이다.

스케줄러는 어떻게 스케줄링 해야 할까?

여담: 선점형 스케줄러

예전 일괄처리 시절에 많은 비선점 스케줄러가 개발되었다.

이 시스템은 각 작업이 종료될 때까지 계속 실행한다.

이론적으로 모든 현대 스케줄러는 선점형이고, 다른 프로세스를 실행시키기 위하여 필요하면 현재 프로세스의 실행을 중단한다.

이 사실은 스케줄러가 앞에서 배웠던 기법을 채용하고 있다는 것을 의미한다.

구체적으로 스케줄러는 문맥 교환을 수행할 수 있고, 실행 중인 프로세스를 잠시 중단시킬 수 있으며 다른 프로세스를 재개 또는 시작시킬 수 있다.

[6] 최소 잔여시간 우선

이 문제를 해결하기 위해서는 가정 3(작업은 끝날 때까지 계속 실행된다)을 완화해야 한다. 이제 그 작업을 해보자.

타이머 인터럽트와 문맥 교환을 고려하면 B와 C가 도착했을 때 스케줄러는 다른 일을 할 수 있다.

작업 A를 중지하고 지금 도착한 B나 C를 실행하기로 결정할 수도 있다.

물론, 선점된 A는 나중에 다시 실행된다.

SJF는 비선점형 스케줄러이기 때문에 이와 같은 동작을 하지 못한다.

다행히도 이렇게 동작할 수 있는 스케줄러는 이미 존재한다.

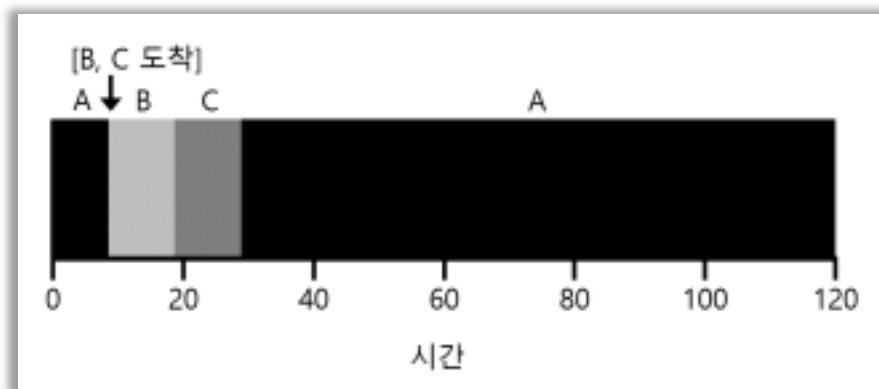
SJF에 선점 기능을 추가한 최단 잔여시간 우선(Shortest Time-to-Completion First, STCF)

또는 선점형 최단 작업 우선(PSJF)으로 알려진 스케줄러이다.

언제든 새로운 작업이 시스템에 들어오면, 이 스케줄러는 남아 있는 작업과 새로운 작업의 잔여 실행 시간을 계산하고 그 중 가장 적은 잔여 실행 시간을 가진 작업을 스케줄한다.

우리의 예에서는 STCF는 A를 선점하고, B와 C를 끝날 때까지 실행시킨다.

두 작업이 모두 끝난 후에 A가 스케줄되어 남은 실행 시간 만큼 실행된다.



그 결과 평균 반환 시간이 단축되어 50 초($\frac{(120-0)+(20-10)+(30-10)}{3}$)가 된다.

새로운 가정 하에서 STCF가 최적의 스케줄러이다.

작업들이 동시에 도착할 경우, SJF가 최적의 결과를 낸다는 것을 고려하면, STCF가 최적의 스케줄링이 되는 이유를 쉽게 알 수 있을 것이다.

[7] 새로운 평가 기준: 응답 시간

작업의 길이를 미리 알고 있고, 작업이 오직 CPU 만 사용하며, 평가 기준이 반환 시간 하나라면, STCF 는 매우 훌륭한 정책이다.

초기 일괄처리 컴퓨터 시스템에서는 이러한 스케줄링 알고리즘이 의미가 있었다.

그러나 시분할 컴퓨터의 등장이 모든 것을 바꾸었다.

이제 사용자는 터미널에서 작업하게 되어 시스템에서 상호작용을 원활히 하기 위한 성능을 요구하게 되었다.

응답 시간(response time)이라는 새로운 평가 기준이 태어나게 된다.

응답 시간은 작업이 도착할 때부터 처음 스케줄 될 때까지의 시간으로 정의된다.

$$T_{\text{응답 시간}} = T_{\text{첫 실행}} - T_{\text{도착 시간}}$$

예를 들어, 위와 같은 스케줄러의(A 는 시간 0, B 와 C 는 시간 10 에 도착) 경우, 각 작업의 응답 시간은 A 는 0, B 는 0, C 는 10, 평균 3.33 이 된다.

이미 생각하고 있는지 모르지만,

STCF 를 비롯하여 비슷한 류의 정책들은 응답 시간이 짧다고 할 수 없다.

예를 들어, 3 개의 작업이 동시에 도착한 경우, 세 번째 작업은 딱 한 번 스케줄 되기 위해 먼저 실행된 두 작업이 완전히 끝날 때까지 기다린다.

반환 시간 기준으로는 매우 훌륭한 반면, 응답 시간과 상호작용 측면에서는 매우 나쁜 방법이다.

터미널 앞에 앉아 입력한 후, 단지 다른 작업이 당신의 작업보다 먼저 스케줄되었다는 이유로 시스템의 응답이 올 때까지 10 초 기다리는 것을 상상해보자.

별로 즐거운 일은 아니다.

[8] 라운드 로빈(Round Robin Scheduling)

응답 시간 문제를 해결하기 위해 라운드 로빈(Round Robin, RR) 스케줄링 알고리즘이 사용된다.

RR의 기본 아이디어는 매우 단순하다.

각 작업이 완료될 때까지 실행하도록 두지 않고, 일정 시간 동안만 CPU를 사용하게 한 뒤 다음 작업으로 전환하는 방식이다.

이때 각 작업이 CPU를 사용하는 일정 시간을 타임 슬라이스(Time Slice) 또는 스케줄링 쿼텀이라고 한다.

작업이 종료되지 않았다면, 다시 실행 큐의 뒤로 이동하여 다음 차례에 다시 실행된다. 이러한 방식 때문에 RR은 타임 슬라이싱(Time Slicing)이라고도 불린다.

타임 슬라이스는 타이머 인터럽트 주기의 배수로 설정된다.

예를 들어, 타이머가 10ms(0.01 초)마다 인터럽트를 발생시키면 타임 슬라이스는 10ms, 20ms 등의 값이 된다.

[8-1] 예시

3개의 작업 A, B, C가 동시에 도착하고 각각 5초의 실행 시간을 가진다고 가정하자.

[SJF 스케줄링]

SJF는 하나의 작업이 끝날 때까지 실행한다.

실행 순서

A → B → C

응답 시간

A = 0

B = 5

C = 10

평균 응답 시간

$(0 + 5 + 10) / 3 = 5$

[RR 스케줄링(타임 슬라이스 = 1 초)]
RR 은 작업을 1 초씩 번갈아 실행한다.

A → B → C → A → B → C ...

응답 시간

A = 0
B = 1
C = 2

평균 응답 시간

$(0 + 1 + 2) / 3 = 1$

즉, RR 은 응답 시간을 크게 줄일 수 있다.

[8-2] 타임 슬라이스의 중요성

타임 슬라이스의 길이는 RR 성능에 매우 큰 영향을 미친다.

타임 슬라이스가 짧아졌을 때

- 장점
 - 응답 시간이 빨라짐
 - 사용자 인터랙션 시스템에 유리
- 단점
 - 문맥 교환(context switch)이 자주 발생
 - 시스템 오버헤드 증가

타임 슬라이스가 길 때

- 장점
 - 문맥 교환 비용 감소
 - CPU 효율 증가
- 단점
 - 응답 시간 증가
 - 사용자 체감 성능 저하

[8-3] 문맥 교환 비용

문맥 교환 비용은 단순히 레지스터 저장/복원만 포함하지 않는다.
다음과 같은 하드웨어 상태도 영향을 받는다.

- CPU 캐시(Cache)
- TLB
- 분기 예측기(Branch Predictor)

프로세스가 바뀌면 이러한 정보들이 초기화되거나 다시 채워져야 하기 때문에 추가적인 성능 비용이 발생한다.

[8-4] 반환 시간 관점에서의 RR

RR 은 응답 시간(Response Time)에서는 매우 좋은 성능을 보인다.
하지만 반환 시간(Turnaround Time) 기준에서는 좋지 않다.

예를 들어, 위의 작업이 RR(1 초 슬라임)에서 실행되면 종료 시간은 다음과 같다.

A = 13
B = 14
C = 15

평균 반환 시간

$$(13 + 14 + 15) / 3 = 14$$

이는 SJF 보다 훨씬 나쁜 결과이다.

[8-5] 스케줄링의 핵심 트레이드오프

CPU 스케줄링에서는 두 가지 성능 목표가 서로 충돌하는 경우가 많다.

1. 반환 시간(Turnaround Time)
2. 응답 시간(Response Time)

이 두 가지를 동시에 최적화하는 것은 어렵기 때문에 스케줄러는 둘 사이의 트레이드오프(trade-off)를 고려하여 설계된다.

반환 시간 최적화의 특징

이러한 알고리즘은 한 작업을 가능한 빨리 끝내는 것이 목표이다.

- CPU 를 한 작업에 집중적으로 할당
- 다른 작업들은 오래 기다릴 수 있음

결과적으로

- 반환 시간은 좋아지지만 응답 시간은 나빠질 수 있다.

응답 시간 최적화의 특징

모든 프로세스가 가능한 빨리 CPU 를 한 번씩 사용하도록 만드는 것

- CPU 가 여러 작업 사이를 계속 전환
- 작업 완료까지 시간이 길어질 수 있음

결과적으로

- 응답 시간은 매우 좋아지지만 반환 시간은 나빠질 수 있다.

[8-6] 비용 상쇄

시스템에서는 어떤 연산을 수행해야할 때 고정 비용(fixed cost)이 발생하는 경우가 많다. 이때 전체 시스템 성능을 개선하기 위해 상쇄(amortization)라는 기법을 사용한다.

상쇄의 기본 아이디어는 다음과 같다.

비용이 발생하는 연산의 실행 횟수를 줄여 전체 평균 비용을 낮추는 것이다.

즉, 한 번의 비용은 그대로 두되 그 비용이 전체 실행에서 차지하는 비율을 줄이는 방식이다.

스케줄링에서의 예시

CPU 스케줄링에서도 이러한 상쇄 기법이 사용된다.

예를 들어, 다음과 같은 상황을 가정하자.

- 타임 슬라이스: 10ms
- 문맥 교환 비용: 1ms

이 경우,

문맥 교환 비율 = $1 \text{ ms} / 10 \text{ ms} = 10\%$

즉, CPU 시간의 약 10%가 문맥 교환에 사용되어 낭비된다.

문맥 교환 비율

$$\text{문맥교환 비율} = \frac{\text{문맥교환시간}}{\text{타임슬라이스}}$$

한 번의 실행 사이클

프로세스 실행 : 10 ms

문맥 교환 : 1 ms

총 시간 : 11 ms

즉,

- 10ms: 실제 작업
- 1ms: 프로세스 교체

비용을 상쇄하는 방법

이 비용을 줄이기 위해 타임 슬라이스를 증가시킬 수 있다.

예를 들어,

- 타임 슬라이스: 100ms
- 문맥 교환 비용: 1ms

이 경우

$$\text{문맥 교환 비율} = 1 \text{ ms} / 100 \text{ ms} = 1\%$$

따라서 문맥 교환이 차지하는 비용이 1% 미만으로 감소하게 됩니다.

즉, 타임 슬라이스를 늘리면 문맥 교환 비용이 전체 실행 시간에서 차지하는 비율이 줄어드는 비용이 상쇄된다.

팁: 중첩(Overlap)은 높은 이용률을 가능하게 한다.

시스템의 자원 활용도를 높이기 위해 여러 작업을 가능한 한 겹치게(중첩되게) 실행하는 전략이 사용된다.

이를 중첩(Overlap)이라고 한다.

중첩의 기본 아이디어는 다음과 같다.

- 어떤 작업이 CPU 를 사용하지 않는 동안, 다른 작업이 CPU 를 사용하도록 하여 전체 시스템 자원의 이용률을 높이는 것이다.

[9] 입출력 연산의 고려

우선 가정 4를 완화하자. 모든 프로그램은 입출력 작업을 수행한다.

아무런 입력도 받아들이지 않는 프로그램을 상상해보자. 매번 같은 출력을 만들어 낼 것이다. 출력하지 않는 프로그램을 상상해보자. 그건 속담에도 나올 법한 아무도 보는 사람이 없이 숲에서 잎이 떨어지는 것과 같다.

실행되었다는 사실이 아무 의미가 없다.

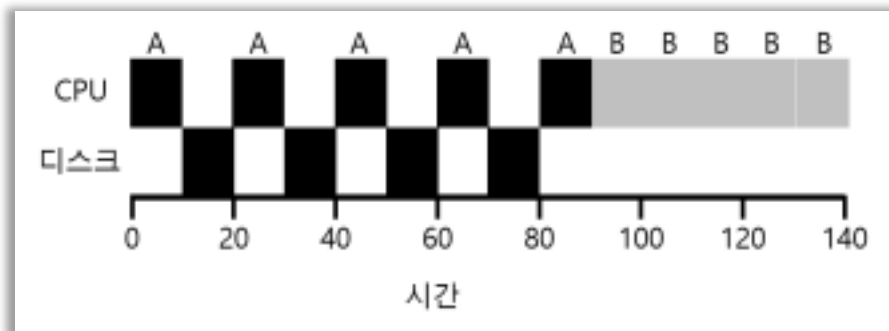
입출력 작업을 요청한 경우 스케줄러는 다음에 어떤 작업을 실행할지를 결정해야 한다. 현재 실행 중인 작업은 입출력이 완료될 때까지 CPU를 사용하지 않을 것이기 때문이다. 입출력 요청을 발생시킨 작업은 입출력 완료를 기다리며 대기 상태가 된다.

입출력이 하드 디스크 드라이브에 보내진 경우, 프로세스는 드라이브의 현재 입출력 워크로드에 따라 몇 초 또는 좀 더 긴 시간 동안 블록될 것이다. 스케줄러는 그 시간 동안 실행될 다른 작업을 스케줄 해야 한다.

마찬가지로 스케줄러는 입출력 완료 시에도 의사 결정을 해야 한다.

입출력이 완료되면 인터럽트가 발생하고 운영체제가 실행되어 입출력을 요청한 프로세스를 대기 상태에서 준비 상태로 다시 이동시킨다.

물론, 인터럽트가 발생했을 때 요청 프로세스를 즉시 실행시키기로 결정할 수도 있다. 운영체제는 각 작업을 어떻게 처리해야 하는가?



이 쟁점을 더 잘 이해하기 위해서 두 개의 작업 A와 B가 있다고 가정한다.

각 작업은 50ms의 CPU 시간을 필요로 한다.

그러나 둘은 한 가지 큰 차이를 가진다.

A는 10ms 동안 실행된 후, 입출력 요청을 한다(여기서는 입출력 시간은 10ms이라고 가정).

반면에 B는 입출력을 수행하지 않는다.

스케줄러는 A를 먼저 실행시키고 B를 다음에 실행시킨다.

STCF 스케줄러를 구축하려고 한다고 가정하자.

A가 5개 10ms 작업으로 분할되는 반면에 B는 하나의 50ms의 CPU를 요청한다는 사실을 어떻게 활용할까?

분명히 입출력을 고려하지 않고 작업을 하나씩 실행시키는 것은 의미가 없다.

일반적인 접근 방식은 A 의 각 10ms 하위 작업을 독립적인 작업으로 다루는 것이다. 시스템이 시작할 때 10ms 작업들과 50ms 를 스케줄하는 것이다.

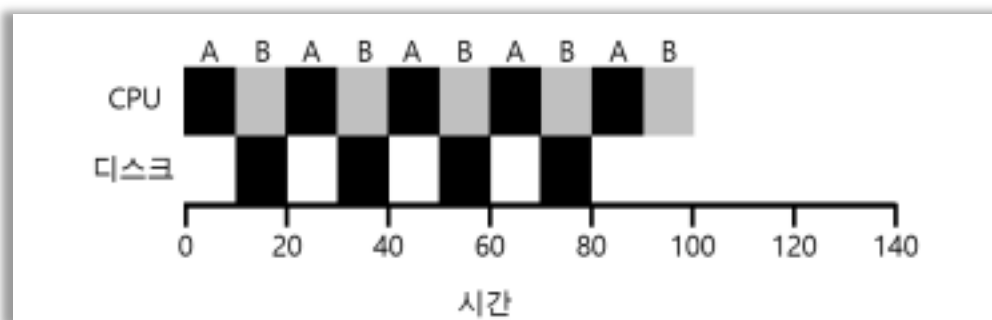
STCF 의 경우 선택은 자명하다. 가장 짧은 작업을 선택하라.

이 경우에는 A 가 된다. 그러면 A 의 첫 번째 소 작업이 완료되면 B 만 남게 되어 실행을 시작한다.

A 의 다음 작업이 제출되고 B 를 선점하여 10ms 동안 실행한다.

이렇게 하면 프로세스의 입출력이 끝나기를 기다리는 동안 CPU 는 다른 프로세스에 의해 사용되어 연산의 중첩이 가능해진다.

시스템은 좀 더 잘 활용된다.



입출력을 고려한 스케줄러 방법에 대해서 알아보았다.

각 CPU 버스트를 하나의 작업으로 간주함으로써 스케줄러는 대화형 프로세스가 더 자주, 즉 유리하게 실행되는 것을 보장한다.

이러한 대화형 작업이 입출력을 실행하는 동안 다른 CPU-집중 작업들이 실행되고 CPU 의 이용률은 더 높아진다.

[1-5] 스케줄링: 멀티 레벨 피드백 큐

핵심 질문: 정보 없이 스케줄 하는 방법은 무엇인가

작업의 실행 시간에 대한 선행 정보 없이 대화형 작업의 응답 시간을 최소화하고 동시에 반환 시간을 최소화하는 스케줄러를 어떻게 설계할 수 있는가?

이 장에서는 멀티 레벨 피드백 큐(Multi-level Feedback Queue, MLFQ)로 알려진 가장 유명한 스케줄링 기법에 대해 논의한다.

MLFQ 가 해결하려고 하는 기본적인 문제는 두 가지이다.

첫째, 짧은 작업을 먼저 실행시켜 반환 시간을 최적화하고자 한다.

이에 대해서는 전 장에서 논의한 바 있다.

SJF 나 STCF 같은 알고리즘은 작업의 실행 시간 정보를 필요로 하지만, 불행히도 운영체제는 이 실행 시간을 미리 알 수 없다.

둘째, MLFQ 는 대화형 사용자(즉, 화면 앞에 앉아 바라보며 프로세스의 종료를 기다리는 사용자)에게 응답이 빠른 시스템이라는 느낌을 주고 싶었기 때문에 응답 시간을 최적화한다.

불행히도 RR 과 같은 알고리즘은 응답 시간을 단축시키지만 반환 시간은 거의 최악이다. 우리의 문제는 다음과 같다.

우리가 프로세스에 대한 정보가 없다면 이러한 스케줄러를 어떻게 만들 수 있을까?

실행 중인 작업의 특성을 알아내고 이를 이용하여 더 나은 스케줄링 결정을 하기 위한 방법은 무엇인가?

[1] MLFQ: 기본 규칙

멀티 레벨 피드백 큐(Multi-Level Feedback Queue, MLFQ)는 여러 개의 큐를 사용하여 프로세스를 스케줄링하는 알고리즘이다.

각 큐에는 서로 다른 우선순위(priority level)가 부여되며, 실행 준비 상태에 있는 프로세스는 이들 중 하나의 큐에 위치하게 된다.

스케줄러는 프로세스를 선택할 때 우선순위를 기준으로 판단하며, 항상 더 높은 우선순위를 가진 큐에 있는 프로세스를 먼저 실행한다.

따라서 높은 우선 순위 큐에 프로세스가 존재하는 동안에는 낮은 우선순위 큐의 프로세스는 실행되지 않는다.

같은 우선순위를 가진 큐 안에는 여러 개의 프로세스가 존재할 수 있으며, 이러한 프로세스들 사이에서는 라운드 로빈(Round-Robin, RR) 방식이 사용된다.

즉, 동일한 우선순위의 프로세스들은 일정한 타임 슬라이스(time slice) 동안 번갈아가며 CPU 를 사용하게 된다.

MLFQ 스케줄링의 핵심 특징은 우선순위가 고정되어 있지 않고 동적으로 변화한다는 점이다. 운영체제는 프로세스의 실행 패턴을 관찰하면서 해당 프로세스의 우선순위를 조정한다.

예를 들어, 어떤 프로세스가 키보드 입력과 같은 입출력을 기다리며 자주 CPU 를 양보한다면 이는 대화형(interactive) 작업일 가능성이 높기 때문에 높은 우선순위를 유지하게 된다. 반대로, 하나의 프로세스가 오랜 시간 동안 CPU 를 계속 사용한다면 CPU 중심 작업(CPU-bound)으로 판단되어 우선순위가 점차 낮아지게 된다.

이와 같이 MLFQ 는 프로세스의 실행 중에 얻은 정보를 기반으로 미래의 행동을 예측하여 스케줄링을 수행한다.

MLFQ 는 다음 두 가지 기본 규칙을 따른다.

규칙 1: $\text{Priority}(A) > \text{Priority}(B)$ 이면 프로세스 A 가 실행되고 B 는 실행되지 않는다.

규칙 2: $\text{Priority}(A) = \text{Priority}(B)$ 이면 두 프로세스는 라운드 로빈 방식으로 번갈아 실행된다.

따라서 시스템에서는 항상 가장 높은 우선순위 큐에 있는 프로세스들이 먼저 실행된다.

예를 들어, 어떤 시점에서 두 개의 프로세스 A 와 B 가 가장 높은 우선순위 큐에 존재하고, 프로세스 C 는 중간 우선순위 큐에, 프로세스 D 는 가장 낮은 우선순위 큐에 존재한다고 가정해보자.

이 경우 스케줄러는 가장 높은 우선순위 큐에 있는 A 와 B 를 라운드 로빈 방식으로 번갈아 실행하게 된다.

그 결과 C 와 D 는 높은 우선순위 큐의 작업이 모두 처리되기 전까지 실행되지 못할 수도 있다.

그러나 이러한 상태는 단순히 특정 시점의 모습일 뿐이며, 실제 MLFQ 에서는 시간이 흐르면서 프로세스의 우선순위가 변화하기 때문에 큐의 구조도 계속 변화하게 된다.

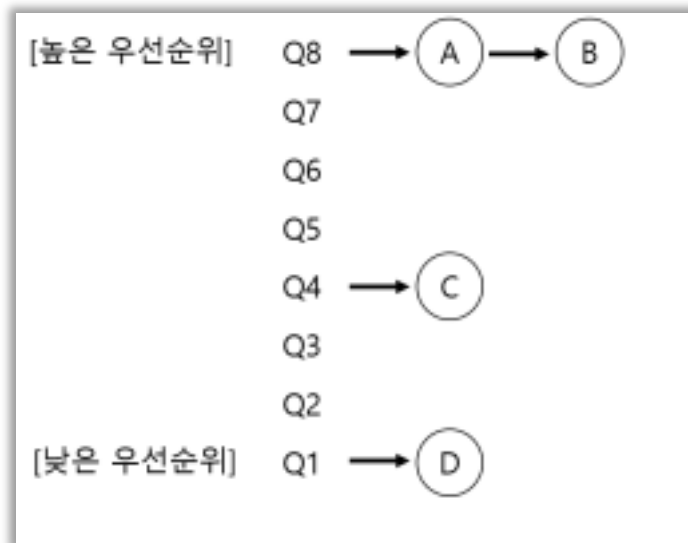
[2] 시도 1: 우선순위 변경

MLFQ 스케줄러는 프로세스 우선순위를 동적으로 조정하여 다양한 작업 특성을 효율적으로 처리하려고 한다.

이를 위해 먼저 프로세스의 특성을 고려해야 한다.

시스템에서는 CPU를 자주 양보하며 빠른 응답이 중요한 대화형 작업(interactive job)과, 긴 시간동안 CPU를 사용하지만 응답 시간이 크게 중요하지 않은 CPU 중심 작업(CPU-bound job)이 함께 존재한다.

이러한 차이를 반영하기 위해 MLFQ는 작업의 실행 패턴에 따라 우선순위를 조정한다.



규칙 3: 작업이 시스템에 진입하면, 가장 높은 우선순위, 즉 맨 위의 큐에 놓여진다.

규칙 4a: 주어진 타임 슬라이스를 모두 사용하면 우선순위는 낮아진다. 즉, 한 단계 아래 큐로 이동한다.

규칙 4b: 타임 슬라이스를 소진하기 전에 CPU를 양도하면 같은 우선순위를 유지한다.

MLFQ의 첫 번째 설계 시도에서는 다음과 같은 규칙을 사용한다.

규칙 3에 따르면 새로운 작업이 시스템에 도착하면 가장 높은 우선순위 큐에 배치된다.

이후 규칙 4a에 따라 프로세스가 주어진 타임 슬라이스를 모두 사용하면 우선순위가 한 단계 낮아지고, 더 낮은 큐로 이동한다.

반대로 규칙 4b에 따르면 타임 슬라이스를 모두 사용하기 전에 CPU를 양도하면 같은 우선순위를 유지한다.

이러한 방식으로 스케줄러는 작업의 실행 패턴을 관찰하면서 점차 해당 작업의 성격을 판단하게 된다.

예 1: 한 개의 긴 실행 시간을 가진 작업

이 작업은 처음에는 가장 높은 우선순위 큐에서 시작하지만, 타임 슬라이스를 계속 모두 사용하게 되므로 실행될 때마다 한 단계씩 낮은 큐로 이동한다.

결국 가장 낮은 우선순위 큐에 도달하게 되고 이후에는 그곳에서 계속 실행된다.

이 과정은 해당 작업이 CPU 중심 작업이라는 것을 점진적으로 드러내는 과정이라고 볼 수 있다.

예 2: 짧은 작업과 함께

CPU 중심 작업 A가 이미 실행 중인 상황에서 짧은 대화형 작업 B가 새로 도착했다고 가정해 보자.

MLFQ는 새로 도착한 작업을 가장 높은 우선순위 큐에 배치하므로 B가 먼저 실행된다.

만약 B가 짧은 실행 시간을 가진 작업이라면 몇 번의 타임 슬라이스만 사용한 뒤 바로 종료하게 된다. 이후에 CPU 중심 작업 A가 다시 실행된다.

이러한 방식으로 MLFQ는 실제 실행 시간을 미리 알 수 없더라도 짧은 작업을 우선적으로 실행하는 SJF 스케줄링과 유사한 효과를 낼 수 있다.

예 3: 입출력 작업에 대해서는 어떻게?

입출력 작업의 경우에도 이러한 규칙이 적용된다.

대화형 작업은 키보드나 마우스 입력을 기다리기 위해 자주 입출력을 수행하므로 타임 슬라이스를 모두 사용하기 전에 CPU를 양도하는 경우가 많다.

이때 규칙 4b에 따라 해당 작업은 같은 우선순위를 유지하게 된다.

따라서 대화형 작업은 계속 높은 우선순위를 유지하며 빠르게 실행될 수 있다.

이는 사용자 입력에 빠르게 반응해야 하는 시스템에서 매우 중요한 특징이다.

하지만 이러한 MLFQ 방식에도 몇 가지 문제점이 존재한다.

첫째, 기아 상태(starvation)가 발생할 수 있다.

시스템에 대화형 작업이 많이 존재하면 높은 우선순위 큐의 작업들이 계속 CPU를 사용하게 되고, 낮은 우선순위 큐에 있는 긴 실행 시간 작업들은 CPU를 거의 할당받지 못할 수 있다.

둘째, 스케줄러를 악용할 가능성이 존재한다.

예를 들어, 어떤 프로그램이 타임 슬라이스가 끝나기 직전에 의도적으로 입출력을 발생시켜 CPU를 양도하면 우선 순위가 낮아지지 않고 높은 큐에 계속 머무를 수 있다.

이렇게 되면 해당 프로그램이 CPU 시간을 과도하게 점유하게 될 수 있다.

마지막으로 프로그램의 특성 시간에 따라 변할 수 있다는 문제도 존재한다.

처음에는 CPU 중심 작업이었던 프로그램이 나중에는 대화형 작업처럼 동작할 수도 있지만, 현재의 방식에서는 이미 낮아진 우선순위를 다시 높이기 어렵기 때문에 이러한 변화를 제대로 반영하지 못한다.

[3] 시도 2: 우선순위의 상향 조정

규칙 5: 일정 시간 S 가 지나면, 시스템의 모든 작업을 최상위 큐로 이동시킨다.

MLFQ 스케줄러의 초기 설계에서는 긴 실행 시간을 가진 CPU 중심 작업이 기아 상태에 빠질 수 있다는 문제가 존재한다.

만약 시스템에 대화형 작업이 계속해서 들어오면, 높은 우선순위 큐의 작업들이 CPU를 계속 차지하게 되고 낮은 우선순위 큐에 있는 긴 작업은 CPU를 거의 할당받지 못하게 된다. 이러한 문제를 해결하기 위해 MLFQ는 우선순위 상향 조정(priority boost)이라는 방법을 도입한다.

우선순위 상향 조정의 기본 아이디어는 일정 시간이 지나면 모든 작업의 우선순위를 다시 높여주는 것이다.

이를 통해 낮은 우선순위 큐에 있던 작업도 다시 높은 우선순위에서 실행될 기회를 얻게 된다.

이를 위해 규칙 5가 추가된다.

이 규칙은 두 가지 중요한 문제를 동시에 해결한다.

첫째, 기아 상태를 방지할 수 있다. 모든 작업이 주기적으로 최상위 큐로 이동하기 때문에 낮은 우선순위 큐에 있던 작업도 결국 CPU를 사용할 기회를 얻게 된다. 이때 최상위 큐에 있는 작업들은 서로 라운드 로빈(RR) 방식으로 CPU를 공유하게 된다.

둘째, 프로그램의 실행 특성의 시간이 지나면서 변하는 경우에도 이를 반영할 수 있다. 예를 들어 처음에는 CPU 중심 작업이었던 프로그램이 이후에 대화형 작업처럼 동작하게 되더라도, 우선순위 상향 조정을 통해 다시 높은 우선순위에서 실행되며 적절한 스케줄링을 받을 수 있게 된다.

예를 들어, 긴 실행 시간을 가진 작업 하나와 두 개의 대화형 작업이 CPU를 두고 경쟁하는 상황을 생각해보자.

만약 우선순위 상향 조정이 없다면 두 개의 대화형 작업이 계속해서 높은 우선순위를 유지하게 되고, 긴 실행 시간 작업은 거의 실행되지 못하는 기아 상태에 빠질 수 있다. 하지만 일정 시간마다 모든 작업을 최상위 큐로 올리는 우선순위 상향 조정이 적용되면 긴 실행 시간 작업도 주기적으로 높은 우선순위를 얻게 되어 CPU를 사용할 수 있게 된다.

그러나 이 방법에도 고려해야 할 문제가 있다.

바로 우선순위 상황이 발생하는 시간 간격 S 를 어떻게 설정할 것인가 하는 문제이다.

시스템 연구자인 John Ousterhout는 이러한 값을 부동 상수(voo-doo constant)라고 부르며, 정확한 값을 결정하기 어렵다는 점을 지적했다.

만약 S 가 너무 크면 긴 실행 시간 작업이 오랫동안 CPU를 얻지 못해 기아 상태가 발생할 수 있다. 반대로 S 가 너무 작으면 대화형 작업이 충분한 CPU 시간을 확보하지 못하게 된다. 따라서 시스템 설계자는 특성에 맞는 적절한 값을 선택해야 한다.

[4] 시도 3: 더 나은 시간 측정

규칙 4: 주어진 단계에서 시간 할당량을 소진하면(CPU 를 몇 번 양도하였는지 상관없이), 우선순위는 낮아진다.(즉, 아래 단계의 큐로 이동한다.)

MLFQ 스케줄러에는 여전히 해결해야 할 문제가 하나 더 존재한다.

바로 사용자가 프로그램을 의도적으로 작성하여 스케줄러를 자신에게 유리하게 조작할 수 있다는 문제이다.

이러한 문제는 기존의 규칙 4a 와 4b 에서 발생한다.

이 규칙에 따르면 프로세스가 타임 슬라이스를 모두 사용하면 우선순위가 낮아지지만, 타임 슬라이스가 끝나기 전에 CPU 를 양도하면 같은 우선순위를 유지할 수 있다.

따라서 어떤 프로그램이 타임 슬라이스가 끝나기 직전에 의도적으로 입출력을 발생시켜 CPU 를 양도하면 우선순위가 낮아지지 않고 계속 높은 우선순위를 유지할 수 있다.

이러한 방식으로 프로세스는 CPU 시간을 과도하게 점유할 수 있다.

이 문제를 해결하기 위해 MLFQ 는 각 우선순위 단계에서 프로세스가 사용한 총 CPU 시간을 측정하는 방식을 도입한다.

스케줄러는 프로세스가 현재 단계에서 사용한 CPU 시간을 누적하여 기록하고, 해당 단계에서 허용된 CPU 시간 할당량을 모두 소진하면 우선순위를 낮춘다.

이때 중요한 점은 CPU 시간을 한 번에 사용했는지, 여러 번 나누어 사용했는지는 고려하지 않는다는 점이다.

즉, 타임 슬라이스를 한 번에 모두 사용하든 여러 번 나누어 사용하든 총 사용 시간이 일정 수준에 도달하면 동일하게 우선순위가 낮아진다.

이러한 방식으로 기존의 규칙 4a 와 4b 는 하나의 규칙으로 통합된다.

규칙 4 에 따르면 프로세스가 주어진 우선순위 단계에서 할당된 CPU 시간을 모두 소진하면, CPU 를 몇 번 양도했는지와 관계없이 우선순위가 낮아지고 더 낮은 큐로 이동하게 된다.

예를 들어, 어떤 프로세스가 스케줄러를 속이기 위해 타임 슬라이스가 끝나기 직전에 반복적으로 입출력을 수행한다고 가정해보자.

기존 규칙에서는 이러한 방식으로 CPU 를 양도하면 우선순위가 유지되기 때문에 높은 우선순위에서 계속 실행될 수 있었다.

그러나 새로운 규칙에서는 프로세스의 총 CPU 사용 시간이 계속 누적되기 때문에 결국 해당 단계의 시간 할당량을 모두 사용하게 되고, 그 결과 우선순위가 낮아지게 된다.

따라서 프로세스는 더 이상 이러한 방법으로 CPU 시간을 독점할 수 없게 된다.

이와 같이 CPU 사용 시간을 누적하여 측정하는 방식은 스케줄러 조작을 방지하고, 모든 프로세스가 보다 공정하게 CPU 시간을 사용할 수 있도록 만든다.

[5] MLFQ 조정과 다른 쟁점들

MLFQ 스케줄링에는 기본 알고리즘 외에도 여러 가지 추가적인 설계 쟁점이 존재한다. 예를 들어, 스케줄러를 설계할 때 몇 개의 큐를 사용할 것인지, 각 큐의 타임 슬라이스를 얼마로 설정할 것인지, 그리고 우선순위 상향 조정을 얼마나 자주 수행할 것인지와 같은 중요한 변수들을 결정해야 한다.

이러한 값들은 시스템의 워크로드 특성에 따라 달라지기 때문에 하나의 정답이 존재하지 않는다.

따라서 실제 시스템에서는 다양한 환경에서의 경험과 지속적인 조정을 통해 적절한 균형점을 찾는 방식으로 설정된다.

많은 MLFQ 구현에서는 큐마다 서로 다른 타임 슬라이스를 사용한다.

일반적으로 높은 우선순위 큐에는 짧은 타임 슬라이스가 할당된다.

이 큐에는 주로 사용자 입력에 빠르게 반응해야 하는 대화형 작업이 존재하기 때문에 짧은 시간 단위로 빠르게 프로세스를 교체하는 것이 효율적이다.

반면 낮은 우선순위 큐에는 긴 실행 시간을 요구하는 CPU 중심 작업이 위치하므로 비교적 긴 타임 슬라이스가 할당된다.

예를 들어, 가장 높은 우선순위 큐에는 약 10ms 정도의 짧은 타임 슬라이스가 주어지고, 우선순위가 낮아질수록 타임 슬라이스가 점차 길어질 수 있다.

실제 운영체제에서도 이러한 개념이 다양한 방식으로 구현되어 있다.

예를 들어, Solaris 운영체제의 시분할 스케줄링 클래스(Time-Sharing, TS)는 MLFQ 기반 스케줄러를 비교적 쉽게 설정할 수 있도록 설정 테이블을 제공한다.

이 테이블에는 프로세스의 우선순위 변화 방식, 각 큐의 타임 슬라이스 길이, 그리고 우선순위 상향 조정이 이루어지는 주기 등이 정의되어 있다.

기본 설정에서는 약 60 개의 큐가 존재하며, 가장 높은 우선순위 큐의 타임 슬라이스는 약 20ms, 가장 낮은 큐는 수백 밀리초 수준으로 점진적으로 증가한다.

또한, 우선순위 상향 조정은 대략 1 초 간격으로 수행된다.

다른 운영체제에서는 동일한 규칙 기반 방식 대신 수학적 공식을 사용하여 우선 순위를 계산하기도 한다.

예를 들어, FreeBSD의 스케줄러는 프로세스가 사용한 CPU 시간을 기반으로 우선순위를 계산하는 감쇠-사용(decay-usage) 알고리즘을 사용한다.

이 방식에서는 프로세스의 CPU 사용량이 시간이 지남에 따라 점차 감소하도록 계산되어, CPU 사용량이 많았던 프로세스도 일정 시간이 지나면 다시 높은 우선순위를 얻을 수 있도록 한다.

이러한 방식은 앞서 설명한 주기적 우선순위 상향 조정과 유사한 효과를 다른 방법으로 구현한 것이다.

마지막으로 실제 스케줄러는 기본적인 CPU 스케줄링 외에도 다양한 기능을 제공한다.

일부 시스템에서는 가장 높은 우선순위를 운영체제 내부 작업을 위해 예약해 두기도 한다.

또한 사용자에게 프로세스의 우선순위를 어느 정도 조정할 수 있는 기능을 제공하기도 한다. 예를 들어, 유닉스 계열 시스템에서는 nice 명령어를 사용하여 프로세스의 우선순위를 높이거나 낮출 수 있으며, 이를 통해 프로세스의 실행 순서에 영향을 줄 수 있다.

팁: 부두 상수 피하기(Ousterhout's Law)

시스템 설계에서는 가능하다면 부두 상수(voo-do constants)를 사용하는 것을 피하는 것이 좋다.

부두 상수란 명확한 이론적 근거 없이 경험적으로 설정된 값들을 의미한다.

예를 들어, 스케줄러에서 타임 슬라이스의 길이나 우선순위 상향 주기와 같은 값들을 정확하게 계산하기 어렵기 때문에 경험에 기반하여 설정되는 경우가 많다.

그러나 이러한 값들을 완전히 제거하기는 쉽지 않다.

실제 시스템에서는 대부분 기본값이 설정된 설정 파일을 제공하고, 시스템 관리자나 사용자가 필요에 따라 값을 조정할 수 있도록 한다.

하지만 현실적으로는 이러한 값들이 대부분 기본값 그대로 사용되는 경우가 많으며, 단지 기본 설정이 실제 환경에서도 잘 동작하기를 기대하는 경우가 많다.

팁: 가능하면 조언을 이용하시오

운영체제는 모든 프로세스의 특성과 요구 사항을 완벽하게 파악할 수 없기 때문에, 사용자나 관리자 시스템에 힌트(hint)를 제공할 수 있는 인터페이스를 제공하는 것이 도움이 된다. 이러한 힌트를 조언이라고 부른다.

운영체제가 이 조언을 반드시 따라야 하는 것은 아니지만, 추가적인 정보를 활용하여 더 나은 결정을 내리는 데 도움을 받을 수 있다.

이러한 조언은 운영체제의 여러 구성 요소에서 활용된다.

예를 들어, CPU 스케줄링에서는 nice 명령어를 사용하여 프로세스의 우선순위를 조정할 수 있으며, 메모리 관리에서는 madvise 시스템 콜을 통해 메모리 사용 패턴에 대한 정보를 제공할 수 있다.

또하나 파일 시스템에서도 프로그램이 접근 패턴에 대한 정보를 제공하면 운영체제가 선반입(prefetching)이나 캐싱(caching) 전략을 더 효율적으로 수행할 수 있다.

이러한 방식은 운영체제가 스스로 모든 것을 예측하려 하기보다, 사용자나 프로그램의 정보를 활용하여 더 효율적인 자원 관리 결정을 내릴 수 있도록 한다.

[6] MLFQ: 요약

이 장에서는 멀티 레벨 피드백 큐(MLFQ) 스케줄링 알고리즘을 살펴보았다.

MLFQ 라는 이름은 여러 개의 큐를 사용하는 멀티 레벨 구조와, 프로세스의 실행 행동을 관찰하여 우선순위를 조정하는 피드백 메커니즘에서 유래한다.

즉, 프로세스가 과거에 어떻게 CPU 를 사용했는지를 관찰하고, 그 정보를 바탕으로 미래의 스케줄링 우선순위를 결정하는 방식이다.

따라서 프로세스의 실행 패턴이 시간에 따라 어떻게 변화하는지에 따라 스케줄러의 동작도 함께 달라지게 된다.

MLFQ 는 다음과 같은 다섯 가지 기본 규칙을 따른다.

- 규칙 1: $Priority(A) > Priority(B)$ 이면 프로세스 A 가 실행되고 B 는 실행되지 않는다.
- 규칙 2: $Priority(A) = Priority(B)$ 이면 두 프로세스는 라운드 로빈 방식으로 번갈아 실행된다.
- 규칙 3: 새로운 작업이 시스템에 들어오면 가장 높은 우선순위 큐에 배치된다.
- 규칙 4: 프로세스가 현재 단계에서 할당된 CPU 시간을 모두 사용하면, CPU 를 몇 번 양도했는지와 관계 없이 우선순위가 한 단계 낮아진다.
- 규칙 5: 일정한 시간 간격 S 이 지나면 시스템에 존재하는 모든 작업의 우선순위를 최상위 큐로 다시 올린다.

MLFQ 스케줄러의 가장 큰 특징은 작업의 실행 시간을 미리 알 필요 없이, 실행 과정에서 관찰된 행동을 기반으로 스케줄링 정책을 동적으로 적용한다는 점이다.

이를 통해 짧게 실행되는 대화형 작업에 대해서는 빠른 응답을 제공하며, 긴 CPU 중심 작업에 대해서도 일정한 CPU 시간을 보장하여 공정성을 유지한다.

결과적으로 MLFQ 는 응답 시간과 반환 시간 사이의 균형을 맞추는 스케줄링 방식이라고 볼 수 있다.

이러한 장점 때문에 MLFQ 는 실제 운영체제에서도 널리 사용된다.

예를 들어, BSD Unix 계열 운영체제와 그 파생 시스템들, Solaris, 그리고 Windows NT 및 이후의 Microsoft Windows 운영체제에서도 기본적인 CPU 스케줄링 구조로 MLFQ 또는 그 변형 알고리즘이 사용되고 있다.

[1-6] 스케줄링: 비례 배분

핵심 질문: 어떻게 CPU 를 정해진 비율로 배분할 수 있는가
특정 비율로 CPU 를 배분하는 스케줄러를 어떻게 설계할 수 있는가?
그렇게 하기 위한 중요한 기법은 무엇인가?
그 기법은 얼마나 효과적인가?

이 장에서는 비례 배분(Proportional Share) 스케줄러, 혹은 공정 배분(fair share)이라고도 하는 유형의 스케줄러에 대해 다루도록 하겠다.

비례 배분의 개념은 간단하다.

반환 시간이나 응답 시간을 최적화하는 대신 스케줄러가 각 작업에게 CPU 의 일정 비율을 보장하는 것이 목적이다.

비례 배분 스케줄링의 좋은 예가 Waldspurger and Weihl[WW94]의 연구다.

추첨 스케줄링(lottery scheduling)으로 알려져 있다.

이 아이디어는 꽤 오래되었다.

기본 아이디어는 매우 간단하다.

다음 실행될 프로세스를 추첨을 통해 결정된다.

더 자주 수행되어야 하는 프로세스는 당첨 기회를 더 많이 준다.

이제 자세함 사항을 알아보자.

[1] 기본 개념: 추첨권이 당신의 몫을 나타낸다(추첨 스케줄링(Lottery Scheduling))

추첨 스케줄링은 추첨권(티켓)이라는 개념을 기반으로 동작하는 CPU 스케줄링 방법이다.

추첨권은 각 프로세스가 받아야 할 CPU 자원의 비율(몫)을 나타낸다.

즉, 어떤 프로세스가 전체 추첨권 중 얼마나 많은 티켓을 가지고 있는지에 따라 CPU 를 사용할 확률이 결정된다.

프로세스가 보유한 티켓 수가 많을수록 CPU 를 사용할 가능성이 높아진다.

예를 들어, 두 개의 프로세스 A 와 B 가 있다고 가정하자.

프로세스 A 가 75 장의 추첨권을 가지고 있고, 프로세스 B 가 25 장의 추첨권을 가지고 있다면 전체 추첨권은 100 장이 된다.

이 경우 목표는 A 에게 CPU 의 약 75%, B 에게 약 25%를 할당하는 것이다.

추첨 스케줄링에서는 타임 슬라이스가 끝날 때마다 하나의 추첨을 수행한다.

스케줄러는 전체 추첨권의 개수를 확인한 후, 무작위로 하나의 티켓 번호를 선택한다.

예를 들어, 0 부터 99 까지의 번호가 존재한다고 할 때, A 는 0~74 번 티켓을 가지고 있고

B 는 75~99 번 티켓을 가지고 있다고 가정한다.

추첨을 통해 선택된 번호가 어느 프로세스의 티켓에 해당하는지에 따라 실행할 프로세스가 결정된다.

예를 들어, 다음과 같은 추첨 결과가 나왔다고 가정하자.

63	85	70	39	76	17	29	41	36	39	10	99	68	83	63	62	43	0	49	49
A	B	A	A	B	A	A	A	A	A	B	A	B	A	A	A	A	A	A	A

이 예에서는 총 20 번의 타임 슬라이스 중에서 B 가 4 번(20%) CPU 를 사용하였다.

이는 목표 비율인 25%보다 약간 낮은 값이다.

이러한 결과가 나타나는 이유는 추첨 스케줄링이 확률 기반 방식이기 때문이다.

따라서 짧은 시간 동안에는 정확한 비율이 나오지 않을 수 있다.

그러나 실행 시간이 길어질수록 통계적으로 원하는 CPU 비율에 가까워진다.

즉, 장기적으로 보면 각 프로세스가 보유한 티켓 비율에 맞게 CPU 자원이 분배되는 특징을 가진다.

팁: 무작위성 이용

추첨 스케줄링의 중요한 특징 중 하나는 무작위성(Randomness)을 활용한다는 점이다.

시스템에서 어떤 결정을 내려야 할 때, 무작위 방식은 단순하면서도 강력한 해결 방법이 될 수 있다.

무작위 방식은 전통적인 결정 방식에 비해 크게 세 가지 장점을 가진다.

첫째, 특정 상황에서 발생하는 최악의 경우를 피할 수 있다.

전통적인 알고리즘은 특정 패턴에서 성능이 급격하게 떨어지는 문제가 발생할 수 있다.

예를 들어, 가상 메모리의 페이지 교체 정책인 LRU(Least Recently Used)는 일반적으로 좋은 성능을 보이지만, 반복적인 순차 접근 패턴이 발생하면 매우 비효율적으로 동작할 수 있다.

반면, 무작위 방식에서는 특정 패턴에 의해 항상 최악의 결과가 발생하는 상황이 거의 나타나지 않는다.

둘째, 구현이 단순하고 관리해야 할 상태 정보가 적다.

전통적인 공정 배분 스케줄링 알고리즘은 각 프로세스가 지금까지 사용한 CPU 시간을 계속 기록하고 관리해야 한다.

하지만 무작위 추첨 방식에서는 이러한 복잡한 기록이 필요 없으며, 각 프로세스가 보유한 추첨권의 개수 정도만 관리하면 된다.

셋째, 결정 과정이 매우 빠르다.

난수 생성만 빠르게 이루어지면, 스케줄러는 즉시 실행할 프로세스를 결정할 수 있다.

이러한 특성 때문에 빠른 의사결정이 필요한 시스템에서도 효율적으로 사용할 수 있다.

실제로 많은 시스템에서는 속도를 높이기 위해 완전히 무작위가 아닌 의사 난수(pseudo-random) 방식을 사용하기도 한다.

이처럼 무작위성을 이용한 방식은 단순성, 효율성, 그리고 특정 상황에서의 안정적인 성능이라는 장점을 제공한다. =

[2] 추첨 기법

추첨 스케줄링에서는 추첨권(ticket)을 활용하여 자원을 배분하는데, 이를 보다 유연하게 사용하기 위해 여러 가지 기법이 사용된다.

첫 번째 기법은 추첨권 화폐(ticket currency)이다.

이 방법은 각 사용자가 자신만의 기준 화폐 단위를 사용하여 추첨권을 자유롭게 할당할 수 있도록 한다.

시스템은 이러한 사용자별 화폐 단위를 자동으로 전역 기본 화폐(global currency)로 변환하여 공정하게 계산한다.

예를 들어, 사용자 A와 B가 각각 100장의 추첨권을 받았다고 가정하자.

사용자 A는 두 개의 작업(A1, A2)을 실행 중이며, 자신의 기준 화폐로 각각 500장의 추첨권을 할당한다.

즉, 총 1000장의 사용자 화폐 중에서 각각 절반을 배분한 것이다.

시스템은 이를 전역 기준으로 변환하여 A1과 A2에 각각 50장의 추첨권을 할당한다.

한편 사용자 B는 하나의 작업(B1)을 실행 중이며 자신의 기준 화폐 10장 중 10장을 모두 할당한다. 시스템은 이를 전역 기준으로 변환하여 100장의 추첨권으로 계산한다.

따라서 전역 기준으로 보면 전체 추첨권은 200장이 되며, 이를 기준으로 추첨이 진행된다.

두 번째 기법은 추첨권 양도(ticket transfer)이다.

이 방법에서는 한 프로세스가 자신이 가진 추첨권을 일시적으로 다른 프로세스에게 전달할 수 있다.

이 기능은 특히 클라이언트-서버 환경에서 유용하게 사용된다.

예를 들어, 클라이언트가 서버에게 작업을 요청할 경우, 클라이언트는 서버에게 자신의 추첨권을 일부 전달하여 서버가 더 많은 CPU 시간을 얻도록 할 수 있다.

이를 통해 요청된 작업이 더 빠르게 처리된다.

작업이 완료되면 서버는 전달받은 추첨권을 다시 클라이언트에게 반환한다.

세 번째 기법은 추첨권 팽창(ticket inflation)이다.

이 방식에서는 프로세스가 일시적으로 자신이 가진 추첨권의 수를 늘리거나 줄일 수 있다.

다만 서로 신뢰하지 않는 프로세스들이 경쟁하는 환경에서는 문제가 발생할 수 있다.

예를 들어, 한 프로세스가 지나치게 많은 추첨권을 자신에게 할당하면 CPU 자원을 독점할 가능성이 있기 때문이다.

따라서 추첨권 팽창 기법은 서로 신뢰할 수 있는 프로세스 환경에서 주로 사용된다.

이러한 경우 특정 프로세스가 더 많은 CPU 시간을 필요로 할 때, 다른 프로세스와 직접 협의하지 않고도 자신의 추첨권 가치를 일시적으로 높여 필요한 자원을 확보할 수 있다.

이와 같은 기법들은 추첨 스케줄링을 더욱 유연하고 효율적인 자원 분배 방식으로 만들어 준다.

팁: 몫을 표현하기 위한 추첨권의 사용

추첨 스케줄링(그리고 보폭 스케줄링)에서 가장 핵심적인 개념은 추첨권이다.

추첨권은 특정 프로세스가 시스템 자원에서 얼마만큼의 몫(share)을 가져야 하는지를 표현하는 데 사용된다.

앞서 설명한 예에서는 추첨권이 CPU 사용 비율을 나타내는데 사용되었지만, 이 개념은 CPU 스케줄링에만 국한되지 않고, 다양한 시스템 자원 관리에도 적용될 수 있다.

예를 들어, 가상화 환경에서 하이퍼바이저(hypervisor)가 여러 가상 머신에게 메모리를 분배할 때도 이러한 개념을 활용할 수 있다.

연구자 Carl A. Waldspurger 는 가상화 환경에서 게스트 운영체제 간의 메모리 분배 비율을 표현하기 위해 추첨권을 사용할 수 있음을 보여주었다.

이 방식에서는 각 게스트 운영체제가 가진 추첨권 수가 곧 메모리 자원에 대한 지분(share)을 의미하게 된다.

이처럼 추첨권은 CPU 뿐만 아니라, 메모리, 네트워크 대역폭, 디스크 I/O 등 다양한 자원 관리 문제에 적용될 수 있는 일반적인 개념이다.

따라서, 시스템에서 어떤 자원의 현재 지분이나 비율을 표현해야 할 필요가 있다면, 추첨권과 같은 방식이 매우 효과적인 해결책이 될 수 있다.

[3] 구현

```
// counter: 당첨 티켓을 찾기 위해 현재까지 누적된 티켓 수
int counter = 0;

// winner: 0 부터 전체 티켓 수 사이의 난수를 생성
// 난수 발생기를 이용하여 당첨 티켓 번호를 선택
int winner = getrandom(0, totaltickets);

// current: 작업 리스트를 순회하기 위한 포인터
node_t *current = head;

// counter 값이 winner 보다 커질 때까지 반복
while(current){
    counter = counter + current->tickets;

    if(counter > winner) break;           // 당첨 프로세스 발견

    current = current->next;
}
// current 가 당첨된 프로세스를 가리킴 -> 해당 프로세스를 실행
```

코드 동작 설명

1. 난수 생성
 - winner = getrandom(0, totaltickets)
 - 전체 티켓 범위 중 하나를 무작위로 선택한다.
2. 리스트 순회
 - current 포인터로 프로세스 리스트를 순차적으로 탐색한다.
3. 티켓 누적
 - 각 프로세스의 티켓 수를 counter 에 계속 더한다.
4. 당첨 프로세스 결정
 - counter > winner 가 되는 순간 해당 프로세스가 당첨된다.
5. CPU 실행
 - current 가 가리키는 프로세스를 실행한다.

추첨 스케줄링의 가장 큰 장점 중 하나는 구현이 매우 단순하다는 점이다.
이 방식을 구현하기 위해 필요한 요소는 크게 세 가지이다.

첫째, 난수 발생기
둘째, 프로세스 집합을 관리하기 위한 자료 구조(예: 리스트)
셋째, 전체 추첨권의 개수

예를 들어, 프로세스를 리스트 형태의 자료 구조로 관리한다고 가정하자.
시스템에는 A, B, C 세 개의 프로세스가 존재하며, 각 프로세스는 서로 다른 수의 추첨권을 가지고 있다.
스케줄러는 먼저 전체 추첨권의 개수를 계산한 뒤, 난수 발생기를 이용하여 그 범위 안에서 하나의 번호를 무작위로 선택한다.

그 다음 리스트에 있는 프로세스를 순서대로 확인하면서 각 프로세스가 보유한 추첨권의 수를 누적해 나간다.
누적된 추첨권 범위 안에 선택된 난수 값이 포함되는 순간, 해당 프로세스가 당첨된 것으로 판단하고 CPU 를 할당한다.

예를 들어, 다음과 같이 추첨권이 분배되어 있다고 가정하자.

- A 장: 10 장
- B 장: 20 장
- C 장: 30 장

전체 추첨권은 60 장이 된다.
이때 난수 발생기가 0 부터 59 사이의 값을 하나 선택한다.
만약 선택된 값이 0~9 범위라면 A 가 실행되고, 10~29 범위라면 B 가 실행되고,
30~59 범위라면 C 가 실행된다.

이와 같은 방식으로 스케줄러는 간단한 자료 구조와 난수 생성만으로도 공정한 자원 분배를 수행할 수 있다.
따라서 구현이 복잡하지 않으면서도, 장기적으로는 각 프로세스가 가진 추첨권 비율에 따라 CPU 시간을 분배할 수 있다는 장점을 가진다.

[4] 추첨 스케줄링의 공정성 분석

추첨 스케줄링의 동작을 쉽게 이해하기 위해 두 개의 작업이 CPU를 공유하는 상황을 생각해 보자.

두 프로세스는 각각 같은 수의 추첨권(100장)을 가지고 있으며, 동일한 실행 시간 R을 가진다고 가정하자.

이 경우 우리의 목표는 두 작업이 거의 동시에 종료되도록 하는 것이다.

하지만 추첨 스케줄링은 무작위(random) 방식으로 실행할 프로세스를 선택하기 때문에, 실제 실행 과정에서는 한 작업이 다른 작업보다 먼저 종료될 수 있다.

이러한 차이를 측정하기 위해 불공정 지표(unfairness metric) U를 정의한다.

이 값은 다음과 같이 계산된다.

$$U = \frac{\text{첫 번째 작업이 종료된 시간}}{\text{두 번째 작업이 종료된 시간}}$$

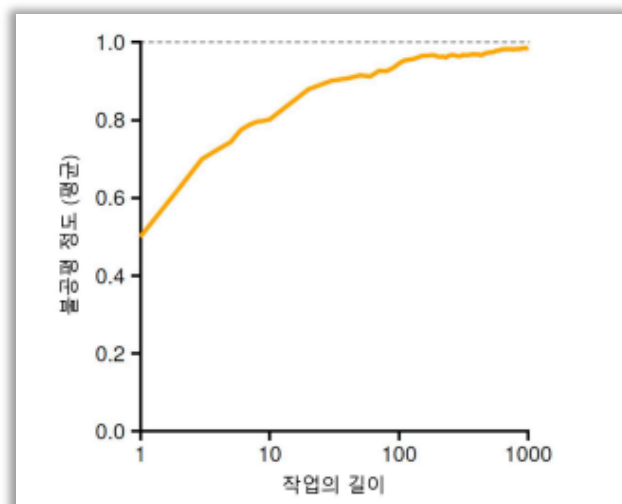
예를 들어, 작업 실행 시간이 R = 10이라고 가정하자.

- 첫 번째 작업 종료 시간: 10
- 두 번째 작업 종료 시간: 20

이 경우 불공정 지표는 다음과 같다.

$$U = \frac{10}{20} = 0.5$$

만약 두 작업이 거의 동시에 종료된다면 U 값은 1에 가까워진다.



따라서 이 실험에서의 목표는 U 값이 1에 최대한 가까워지는 것이다.

이론적으로 완벽하게 공정한 스케줄러라면 두 작업이 동시에 종료하므로 U = 1이 된다.

실험 결과를 보면, 작업의 실행 시간이 짧을 경우 평균적인 불공정 정도가 상당히 크게 나타난다.

이는 추첨 스케줄링이 확률 기반 방식이기 때문에 짧은 시간에서는 무작위성의 영향이 크게 나타나기 때문이다.

반면 작업이 충분히 긴 시간 동안 실행될 경우, 통계적인 효과로 인해 추첨 스케줄러는 점차 원하는 CPU 분배 비율에 가까워지게 된다.

즉, 실행 시간이 길어질수록 두 작업의 종료 시점도 점점 비슷해지고 불공정 지표 U 역시 1에 가까워지는 경향을 보인다.

이 실험 결과는 추첨 스케줄링이 단기적으로 불공정할 수 있지만 장기적으로는 공정성을 달성하는 특성을 가진다는 점을 보여준다.

[5] 추첨권 배분 방식

추첨 스케줄링에서 아직 해결되지 않은 중요한 문제 중 하나는 추첨권을 각 작업에 어떻게 배분할 것인가 이다.

즉, 각 작업에게 몇 장의 추첨권을 할당해야 하는지를 결정하는 문제이다.

이 문제는 시스템의 동작 방식에 큰 영향을 미치기 때문에 쉽게 결정하기 어렵다.

한 가지 가능한 접근 방식은 사용자가 자신의 작업 특성을 가장 잘 알고 있다고 가정하는 것이다.

이 방법에서는 시스템이 먼저 각 사용자에게 일정 수의 추첨권을 할당하고, 그 다음 사용자가 자신이 실행시키고자 하는 작업들 사이에서 자율적으로 추첨권을 분배하도록 한다.

그러나 이러한 방법은 근본적인 해결책이라고 보기는 어렵다.

단순히 사용자에게 권한을 넘겨주는 것일 뿐, 여러 기준으로 추첨권을 배분해야 하는지에 대한 명확한 지침을 제공하지 못하기 때문이다.

결국 특정 작업 집합이 주어졌을 때 추첨권을 어떻게 공정하고 효율적으로 배분할 것인가라는 문제는 여전히 해결되지 않은 과제로 남아 있다.

따라서 추첨 스케줄링에서는 추첨 방식 자체는 비교적 단순하지만, 실제 시스템에서 적용하기 위해서는 추첨권 배분 정책을 어떻게 설계할 것인가가 중요한 연구 주제로 남아 있다.

[6] 왜 결정론적(Deterministic) 방법을 사용하지 않는가

앞에서 살펴본 것처럼 무작위성을 사용하는 추첨 스케줄링은 구현이 단순하고 효율적이지만, 항상 정확한 CPU 비율을 보장하지는 못한다.

특히 작업이 짧은 시간 동안만 실행되는 경우에는 무작위성의 영향 때문에 원하는 비율과 실제 CPU 사용 비율 사이에 차이가 발생할 수 있다.

이러한 문제를 해결하기 위해 연구자인 Carl A. Waldspurger는 결정론적 공정 배분 스케줄러인 보폭 스케줄링(strid scheduling)을 제안하였다.

보폭 스케줄링은 추첨 스케줄링과 달리 확률이 아닌 계산을 통해 정확한 CPU 비율을 보장하는 방식이다.

보폭 스케줄링에서는 각 프로세스가 보폭(strid)이라는 값을 가진다.

이 값은 프로세스가 가진 추첨권 수에 반비례한다.

예를 들어, 프로세스 A, B, C가 각각 100, 50, 250장의 추첨권을 가진다고 가정하자.

큰 상수 값(예: 10,000)을 각 추첨권 수로 나누면 다음과 같은 보폭 값이 계산된다.

프로세스	추첨권	보폭(stride)
A	100	100
B	50	200
C	250	40

각 프로세스는 또한 pass 값을 가지고 있으며, 처음에는 모두 0으로 시작한다.

스케줄러는 pass 값이 가장 작은 프로세스를 선택하여 실행한다.

프로세스가 한 번 실행될 때마다 해당 프로세스의 pass 값에 자신의 stride 값을 더한다.

이를 통해 프로세스가 지금까지 얼마나 CPU를 사용했는지를 추적한다.

보폭 스케줄링의 기본 동작은 다음과 같은 의사 코드로 표현할 수 있다.

```
current = remove_min(queue);    // pass 값이 가장 작은 프로세스 선택
schedule(current);              // 해당 프로세스 실행
current->pass += current->stride; // pass 값을 stride 만큼 증가
insert(queue, current);         // 다시 큐에 삽입
```

예를 들어, A, B, C의 stride 값이 각각 100, 200, 40이고 pass 값이 모두 0에서 시작한다고 하자.

처음에는 세 프로세스의 pass 값이 같으므로 임의의 하나를 선택할 수 있다.

A가 먼저 실행되면 pass 값은 100이 된다.

다음에는 B가 실행되어 pass 값이 200이 되고, 이후 C가 실행되어 pass 값이 40이 된다.

이후에는 항상 pass 값이 가장 작은 프로세스가 선택되면서 실행 순서가 반복된다.

이 과정을 계속 수행하면 실행 횟수는 다음과 같은 비율로 나타난다.

- C: 5 번 실행
- A: 2 번 실행
- B: 1 번 실행

이 실행 횟수는 각 프로세스의 추첨권 수 250: 100: 50:과 정확히 비례한다.

즉, 추첨 스케줄링은 확률적으로 CPU 를 배분하는 반면, 보폭 스케줄링은 매 스케줄링 단계에서 정확한 비율로 CPU 를 배분한다.

그렇다면 보폭 스케줄링이 더 정확한데 왜 추첨 스케줄링을 사용할까?

그 이유는 추첨 스케줄링이 더 단순하고 유연하기 때문이다.

보폭 스케줄링은 각 프로세스의 pass 값과 CPU 사용 상태를 계속 관리해야 하는 부담이 있다.

예를 들어, 스케줄링 도중에 새로운 프로세스가 시스템에 들어온다면, 그 프로세스의 pass 값을 어떻게 설정할 것인지가 문제가 되니.

만약 pass 값을 0 으로 설정하면 새 프로세스가 CPU 를 독점할 가능성이 있다.

반면 추첨 스케줄링에서는 이러한 상태 정보를 유지할 필요가 거의 없다.

새로운 프로세스가 추가될 경우 단순히 해당 프로세스의 추첨권 수와 전체 추첨권 수만 갱신하면 되므로, 시스템에 프로세스를 쉽게 추가할 수 있다.

이러한 단순성과 유연성 때문에 추첨 스케줄링은 실제 시스템 설계에서 여전히 유용하게 사용될 수 있다.

[7] 요약

이 장에서는 비례 배분 스케줄링(proportional-scheduling)의 개념을 소개하고, 이를 구현하는 두 가지 방법인 추첨 스케줄링(lottery scheduling)과 보폭 스케줄링(stride scheduling)을 살펴보았다.

추첨 스케줄링은 무작위성(randomness)을 이용하여 CPU 자원을 배분하는 방식이다. 각 프로세스는 일정 수의 추첨권을 가지고 있으며, 스케줄러는 무작위로 하나의 티켓을 선택하여 실행할 프로세스를 결정한다.

반면 보폭 스케줄링은 결정론적 방식(deterministic method)을 사용한다.

각 프로세스가 가진 추첨권 수를 보폭(stride)을 계산하고, pass 값을 이용해 CPU 사용량을 추천하면서 항상 가장 적은 pass 값을 가진 프로세스를 실행한다.

이를 통해 원하는 CPU 비율을 보다 정확하게 유지할 수 있다.

이 두 방식은 개념적으로 매우 흥미로운 스케줄링 방법이지만, 실제 운영체제의 기본 CPU 스케줄러로 널리 사용되지는 않는다.

그 이유 중 하나는 이러한 방식이 입출력 작업(I/O)과 함께 동작할 때 예상대로 작동하지 않을 수 있기 때문이다.

또한 추첨권을 각 작업에 어떻게 할당해야 하는지에 대한 문제도 완전히 해결되지 않은 상태이다.

예를 들어, 웹 브라우저와 같은 응용 프로그램에 얼마만큼의 추첨권을 할당해야 하는지를 결정하는 것은 쉽지 않다.

이러한 이유로 실제 시스템에서는 MLFQ와 같은 범용 스케줄링 알고리즘이나 Linux 계열의 유사 스케줄러들이 더 널리 사용된다.

이러한 스케줄러들은 다양한 작업 특성을 보다 직관적으로 처리할 수 있기 때문이다.

그럼에도 불구하고 비례 배분 스케줄링은 특정 환경에서 매우 유용하게 사용될 수 있다.

예를 들어 가상화된 데이터 센터에서 특정 가상 머신에 CPU 자원의 일정 비율을 명확하게 할당해야 하는 경우에는 이러한 방식이 간단하고 효과적인 해결책이 될 수 있다.

예를 들어, 한 시스템에서 Windows 가상 머신에 CPU의 1/4을 할당하고 나머지를 Linux 시스템에 배분하는 경우 비례 배분 방식이 적합하다.

이와 같은 방식은 가상화 플랫폼에서도 활용되며, 예를 들어, VMware ESX와 같은 시스템에서는 메모리 자원을 비례적으로 배분하는 데 이러한 개념이 사용된다.

이러한 연구자는 앞서 언급한 Carl A Waldspurger의 연구에서도 자세히 설명되어 있다.

[1-7] 스케줄링: 멀티프로세서 스케줄링(고급)

핵심 질문: 여러 CPU 에 작업을 어떻게 스케줄 해야 하는가
운영체제는 어떻게 작업을 여러 CPU 에 스케줄 해야 하는가?
어떤 새로운 문제가 등장하는가?
예전 기술을 적용할 것인가 아니면 새로운 아이디어가 필요한가?

이 장에서는 멀티프로세서 스케줄링의 기본 개념을 소개한다.

이 주제는 비교적 고급 내용에 해당하기 때문에, 보통 병행성(concurrency) 개념을 어느정도 학습한 후에 이해하는 것이 좋다.

병행성은 운영체제에서 매우 중요한 주제이며 이후 장에서 자세히 다루어진다.

과거에는 멀티프로세서 시스템이 고성능 서버나 슈퍼컴퓨터에서만 사용되었지만, 현재는 상황이 크게 달라졌다.

오늘날의 데스크톱, 노트북, 스마트폰 대부분이 멀티코어(multicore) 프로세서를 사용한다. 멀티코어 프로세서는 하나의 칩 안에 여러 개의 CPU 코어를 포함하고 있어서 동시에 여러 작업을 처리할 수 있다.

이러한 변화의 가장 큰 이유는 단일 코어 CPU 의 성능 향상이 물리적 한계에 가까워졌기 때문이다.

클럭 속도를 계속 높이는 방식으로는 성능 향상이 어렵게 되었고, 대신 여러 개의 코어를 사용하여 병렬로 처리하는 방식이 새로운 향상 방법으로 자리 잡게 되었다.

겉보기에는 CPU 가 많아지면 프로그램이 더 빠르게 실행될 것처럼 보인다.

그러나 실제로는 그렇지 않은 경우가 많다.

그 이유는 전통적인 응용 프로그램이 하나의 CPU 만 사용하도록 작성되어 있기 때문이다. 예를 들어, 일반적인 C 프로그램은 기본적으로 단일 실행 흐름(single thread)으로 동작한다. 따라서 CPU 를 여러 개 추가하더라도 프로그램은 여전히 하나의 코어만 사용하게 된다.

이 문제를 해결하려면 프로그램을 병렬(parallel)로 실행되도록 다시 작성해야 한다.

보통 스레드(thread)를 사용하여 작업을 여러 부분으로 나누고, 각각을 다른 CPU 에서 동시에 실행하도록 만든다.

이렇게 작성된 멀티쓰레드 프로그램(multithreaded program)은 여러 CPU 코어를 활용할 수 있기 때문에 더 높은 성능을 얻을 수 있다.

멀티코어 환경이 등장하면서 운영체제도 새로운 문제에 직면하게 되었다.

바로 멀티프로세서 스케줄링이다.

지금까지 우리는 단일 CPU 환경에서의 스케줄링 알고리즘(예: FCFS, SJF, MLFQ 등)에 대해 살펴보았다.

그러나 CPU 가 여러 개 존재하는 환경에서는 다음과 같은 새로운 질문이 발생한다.

- 여러 CPU 에 프로세스를 어떻게 배치할 것인가?
- 특정 프로세스를 어떤 CPU 에서 실행하도록 할 것인가?
- CPU 사이에서 프로세스를 이동시켜야하는가?
- 여러 CPU 가 동시에 실행될 때 공정성과 성능을 어떻게 유지할 것인가?

즉, 멀티프로세서 환경에서는 단순히 “다음에 어떤 프로세스를 실행할 것인가”뿐만 아니라 “어떤 CPU 에서 실행할 것인가”까지 고려해야 한다는 점에서 스케줄링 문제가 더 복잡해진다.

다음 내용에서는 이러한 멀티프로세서 환경에서 발생하는 새로운 문제들과 해결 방법들을 살펴보게 된다.

[1] 배경: 멀티프로세서 구조

멀티프로세서 스케줄링에서 발생하는 새로운 문제를 이해하려면 단일 CPU 시스템과 멀티 CPU 시스템의 하드웨어 구조 차이를 먼저 이해해야 한다.

특히 중요한 차이는 CPU 간 데이터 공유 방식과 하드웨어 캐시 사용 방식이다.

[1-1] 단일 CPU 시스템과 캐시

단일 CPU 시스템에서는 보통 캐시(cache) 계층 구조가 존재한다.

캐시는 프로그램 실행 속도를 높이기 위해 사용되는 작고 빠른 메모리이다.

메인 메모리는 모든 데이터를 저장하지만 접근 속도가 느리다.

따라서 자주 사용하는 데이터를 캐시에 복사해 두면 시스템은 느린 메모리를 빠른 메모리처럼 사용할 수 있다.

예를 들어, 프로그램이 load 명령어를 통해 데이터를 읽는 상황을 생각해 보자.

- CPU는 작은 캐시(예: 64KB)와 큰 메모리를 가지고 있다.
- 프로그램이 처음 데이터를 읽을 때는 메인 메모리에서 데이터를 가져오기 때문에 시간이 오래 걸린다.
- 이후 같은 데이터를 다시 읽으면 캐시에 저장된 데이터를 사용하므로 훨씬 빠르게 접근할 수 있다.

메모리 접근 시간의 차이는 매우 크다.

- 메인 메모리 접근: 수십 ~ 수백 나노초
- 캐시 접근: 몇 나노초

이러한 속도 차이 때문에 캐시는 프로그램 성능에 매우 중요한 역할을 한다.

[1-2] 캐시가 잘 동작하는 이유: 지역성(Locality)

캐시는 지역성이라는 특성을 기반으로 동작한다.

지역성에는 두 가지 종류가 있다.

(1) 시간 지역성(Temporal Locality)

한 번 사용된 데이터는 가까운 미래에 다시 사용될 가능성이 높다.

예시

- 반복문에서 계속 사용되는 변수
- 같은 명령어를 반복 실행하는 루프

(2) 공간 지역성(Spatial Locality)

어떤 주소 x 의 데이터를 사용하면 x 주변의 데이터도 곧 사용될 가능성이 높다.

예시

- 배열을 순차적으로 접근하는 프로그램
- 명령어가 메모리에 연속적으로 저장되어 순차 실행되는 경우

이러한 특성 덕분에 하드웨어는 캐시에 어떤 데이터를 저장해야 할지 비교적 정확하게 예측할 수 있고, 캐시는 매우 효율적으로 작동한다.

[1-3] 멀티프로세서 시스템에서의 문제

문제는 여러 CPU가 하나의 메인 메모리를 공유하는 멀티프로세서 시스템에서 발생한다. 다음과 같은 상황을 생각해 보자.

1. CPU 1이 주소 A의 데이터를 읽는다.
2. 해당 데이터가 캐시에 없으므로 메인 메모리에서 값 D를 가져온다.
3. CPU 1은 캐시에 있는 값을 $D \rightarrow D'$ 로 수정한다.
4. 그러나 메인 메모리는 바로 갱신하지 않고 나중에 기록(write-back)한다.

이때 운영체제가 프로세스를 CPU 2로 이동시키면 문제가 발생한다.

5. CPU 2가 다시 주소 A를 읽는다.
6. CPU 2 캐시에 데이터가 없으므로 메인 메모리에서 값을 가져온다.
7. 하지만 메모리에는 아직 옛 값 D가 남아 있다.

즉, CPU 2는 최신 값 D' 이 아니라 오래된 값 D를 읽게된다.

이 문제를 캐시 일관성 문제(cache coherence problem)라고 한다.

[1-4] 캐시 일관성 문제 해결

이 문제는 주로 하드웨어 수준에서 해결된다.

하드웨어는 메모리 주소의 변화를 지속적으로 검사하여 여러 CPU 캐시에 있는 데이터가 항상 일관된 상태를 유지하도록 관리한다.

대표적인 방법이 버스 스누핑(bus snooping)이다.

이 방식에서는

- 각 CPU 캐시가 메모리 버스의 통신을 계속 감시
- 다른 CPU가 특정 데이터를 수정하면

다음과 같은 동작이 발생한다.

(1) invalidate: 다른 캐시에 있는 동일한 데이터 복사본을 무효화

또는

(2) update: 캐시에 있는 데이터를 새로운 값으로 갱신

이 과정을 통해 여러 CPU가 같은 메모리 데이터를 일관된 상태로 유지할 수 있다.

다만 write-back 캐시에서는 메인 메모리에 쓰기가 지연되기 때문에 캐시 일관성 관리가 더욱 복잡해진다.

[2] 동기화를 잊지 마시오

캐시 일관성(cache coherence)이 하드웨어에 의해 보장된다면, 프로그램이나 운영체제는 공유 데이터 접근에 대해 신경쓰지 않아도 될까?
안타깝게도 답은 아니다.

캐시 일관성은 각 CPU 캐시에 있는 데이터 값이 서로 일치하도록 보장할 뿐, 프로그램의 연산 순서나 동시 접근 문제까지 해결해 주지는 않는다.
따라서 여러 CPU가 동시에 같은 데이터를 수정할 경우에는 여전히 동기화(synchronization)가 필요하다.

이 문제는 운영체제에서 매우 중요한 주제인 병행성(concurrency)와 관련되며, 일반적으로 락(lock)과 같은 상호 배제(mutual exclusion) 기법을 사용해 해결한다.

[2-1] 공유 데이터 접근 문제

여러 CPU가 같은 데이터 구조를 동시에 수정하면 잘못된 결과가 발생할 수 있다.
예를 들어, 여러 CPU가 동시에 사용하는 공유 큐(queue) 또는 리스트(list)가 있다고 가정해 보자.
캐시 일관성이 보장된다고 하더라도, 동시에 삽입(insert)이나 삭제(delete)가 수행되면 데이터 구조가 깨질 수 있다.

따라서 이런 자료구조는 원자적(atomic)으로 업데이트되어야 하며, 이를 위해 락(lock)이 필요하다.

[2-2] 예시: 연결 리스트에서 노드 삭제

다음 코드는 연결 리스트에서 첫 번째 노드를 제거(pop)하는 간단한 함수이다.

```
typedef struct __Node_t {
    int value;
    struct __Node_t *next;
} Node_t;

int List_Pop() {
    Node_t *tmp = head;      // 현재 head 저장
    int value = head->value;  // 값 읽기
    head = head->next;        // head 를 다음 노드로 이동
    free(tmp);               // 이전 head 메모리 해제
    return value;            // 값 반환
}
```

이 코드는 단일 CPU 환경에서는 문제없이 동작한다.
하지만 두 개의 CPU가 동시에 실행하면 문제가 발생한다.

[2-3] 문제 상황(Race Condition)

두 개의 스레드가 동시에 이 함수를 실행한다고 가정해 보자.

상황 진행

Thread 1

```
temp = head
```

현재 head 값을 tmp 에 저장

Thread 2

```
tmp = head
```

같은 값을 자신의 tmp 에 저장

각 스레드는 자기 스택에 tmp 를 가지고 있으므로 서로 영향을 받지 않는다.
하지만 둘 다 같은 head 노드를 삭제하려고 한다.

결과적으로 다음과 같은 문제가 발생한다.

- 같은 노드를 두 번 삭제
- 같은 값을 두 번 반환
- 리스트 구조가 깨질 가능성

이러한 문제를 Race Condition(경쟁 상태)라고 한다.

[2-4] 해결 방법: Lock 사용

이 문제를 해결하려면 락(lock)을 사용해 한 번에 하나의 스레드만 리스트를 수정하도록 해야 한다.

예를 들어, mutex 를 사용할 수 있다.

```
pthread_mutex_t m;
```

```
int List_Pop() {  
    lock(&m);  
  
    Node_t *tmp = head;  
    int value = head->value;  
    head = head->next;  
    free(tmp);  
  
    unlock(&m);  
    return value;  
}
```

- 한 스레드가 리스트를 수정하는 동안 다른 스레드는 기다리게 된다.
즉, 상호 배제(mutual exclusion)가 보장된다.

[2-5] 그러나 새로운 문제: 성능

락은 올바른 동작을 보장하지만 성능 문제를 일으킬 수 있다.

특히 멀티프로세서 시스템에서 CPU 수가 많아질수록 문제가 커진다.

- 여러 CPU가 동시에 같은 락을 기다림
- 락을 획득하기 위해 경쟁이 발생
- 결과적으로 병렬성이 감소

즉, CPU가 많아질수록 동기화된 자료구조 접근은 오히려 느려질 수 있다.

[3] 마지막 문제점: 캐시 친화성(Cache Affinity)

멀티프로세서 스케줄링에서 고려해야 할 또 하나의 중요한 문제는 캐시 친화성이다.

[3-1] 캐시 친화성의 기본 개념

프로세스가 특정 CPU 에서 실행되면 그 과정에서 다음과 같은 상태 정보가 해당 CPU 에 저장된다.

- CPU 캐시(cache)안에 데이터
- TLB(Translation Lookaside Buffer)안의 주소 변환 정보

즉, 프로세스가 실행되는 동안 해당 CPU 내부에 많은 관련 정보가 저장된다.

이 상태에서 같은 프로세스가 다시 실행될 때 동일한 CPU 에서 실행된다면 이미 캐시에 필요한 데이터가 일부 존재할 가능성이 높다.

따라서 프로그램을 더 빠르게 실행할 수 있다.

[3-2] CPU 가 바뀌면 생기는 문제

만약 프로세스가 매번 다른 CPU 에서 실행된다면 다음과 같은 일이 발생한다.

1. 새 CPU 의 캐시에 해당 프로세스의 데이터가 없음
2. 메모리에서 데이터를 다시 가져와야 함
3. 캐시를 새로 채워야함

즉, 캐시를 다시 채우는 비용이 발생하여 프로그램 성능이 떨어지게 된다.

[3-3] 캐시 일관성은 문제를 해결하지 않는다.

멀티프로세서 시스템에서는 캐시 일관성 프로토콜이 존재하기 때문에, 프로세스가 다른 CPU 에서 실행되더라도 프로그램의 실행 결과는 올바르게 유지된다.

하지만 중요한 점은 다음과 같다.

- 정확성(correctness)은 유지된다.
- 성능(performance)은 떨어질 수 있다.

따라서 스케줄러는 단순히 어떤 프로세스를 실행할것인지만 고려해서는 안되고, 어느 CPU 에서 실행할 것인지도 고려해야 한다.

[3-4] 스케줄러의 전략

멀티프로세서 스케줄러는 가능한 한 다음 전략을 사용한다.

가능하면 프로세스를 이전에 실행되었던 CPU 에서 다시 실행한다.

- 캐시 데이터를 재사용 가능
- TLB 정보 재사용 가능
- 메모리 접근 감소
- 실행 속도 감소

이것이 바로 캐시 친화성을 고려한 스케줄링이다.

[4] 단일 큐 스케줄링(SQMS, Single Queue Multiprocessor Scheduling)

멀티프로세서 스케줄링을 구현하는 가장 단순한 방법은 기존 단일 CPU 스케줄러를 그대로 사용하는 것이다.

이 방법을 단일 큐 멀티프로세서 스케줄링이라고 한다.

[4-1] 기본 구조

- 시스템 전체에 하나의 스케줄링 큐만 존재
- 모든 CPU가 같은 큐에서 작업을 가져옴

작업 큐

[A] [B] [C] [D] [E]

CPU가 2개라면

CPU0 → A

CPU1 → B

다음 스케줄링에서는

CPU0 → C

CPU1 → D

이런식으로 같은 큐에서 작업을 선택한다.

[4-2] SQMS의 장점

구현이 매우 단순

- | |
|--|
| <ul style="list-style-type: none">- 기존 단일 CPU 스케줄러를 그대로 확장 가능<ul style="list-style-type: none">- Round Robin- MLFQ- 같은 알고리즘을 큰 수정 없이 사용 가능 |
|--|

[4-3] SQMS 의 문제점

확장성 문제

모든 CPU 가 같은 큐를 접근하므로 락(lock)이 필요하다.

```
lock(queue)
    작업 선택
unlock(queue)
```

CPU 수가 증가하면

- 락 경쟁 증가
- 스케줄러가 락 기다리는 시간 증가
- 실제 작업 수행 시간 감소

즉, 성능이 크게 떨어진다.

캐시 친화성 문제

SQMS 에서는 CPU 가 항상 같은 작업을 실행하지 않는다.

작업

```
A B C D E
```

CPU 4 개의 시간 흐름

```
CPU0 : A → E → C
CPU1 : B → A → D
CPU2 : C → B → E
CPU3 : D → C → A
```

문제는 프로세스가 CPU 를 계속 이동한다는 것이다.

- 캐시 재사용 불가능
- 캐시 miss 증가
- 성능 저하

해결 방법(부분해결)

대부분 SQMS 는 다음 정책을 사용한다.

가능하면 같은 CPU 에서 다시 실행

CPU0 : A CPU1 : B CPU2 : C CPU3 : D
--

남은 작업

E

E 만 CPU 사이에서 이동

- | |
|--|
| <ul style="list-style-type: none">- 대부분 작업은 캐시 유지- 일부만 이동 |
|--|

하지만 구현이 복잡해진다.

[4-4] SQMS 핵심정리

장점

- | |
|---|
| <ul style="list-style-type: none">- 구현 간단 |
|---|

단점

- | |
|---|
| <ul style="list-style-type: none">- 락 경쟁- 확장성 부족- 캐시 친화성 문제 |
|---|

그래서 등장한 방법이 다음에 설명하는 멀티 큐 스케줄링이다.

[5] 멀티 큐 스케줄링(MQMS, Multi Queue Multiprocessor Scheduling)

SQMS 의 문제를 해결하기 위해 CPU 마다 큐를 하나씩 두는 방식이 등장했다.

[5-1] 기본 구조

각 CPU 마다 자신의 큐

CPU 2 개라면

CPU0 Queue : A B

CPU1 Queue : C D

각 CPU 는 자기 큐만 사용한다.

스케줄링

CPU0 : A → B → A → B

CPU1 : C → D → C → D

[5-2] MQMS 장점

확장성이 좋음

큐가 분리되어 있기 때문에

- 락 경쟁 거의 없음
- CPU 증가해도 성능 유지

캐시 친화성 좋음

작업이 같은 CPU 에서 계속 실행

CPU0 → A B

CPU1 → C D

캐시 재사용 가능(성능 향상)

[5-3] MQMS 문제점

Load Imbalance(워크로드 불균형)

초기

CPU0 : A C

CPU1 : B D

C 종료

CPU0 : A

CPU1 : B D

결과

CPU0 : A

CPU1 : B → D → B → D

A 는 CPU 를 많이 사용하는 불공정이 발생한다.

더 심한 경우

A 와 C 종료

CPU0 : -

CPU1 : B D

결과

CPU0 : idle

CPU1 : B → D

CPU 하나가 놀게 된다.

[5-4] 해결 방법: 작업 이주(Migration)

작업을 CPU 사이에서 이동

CPU0 : -

CPU1 : B D

이동

CPU0 : B

CPU1 : D

[5-5] Work Stealing(작업 훔치기)

일이 없는 CPU 가 다른 CPU 에서 작업을 가져온다.

CPU0 : -

CPU1 : B D

CPU 0 이

B 훔침

결과

CPU0 : B

CPU1 : D

[5-6] Work Stealing 문제

- 큐를 너무 자주 검사하기에 오버헤드 증가
- 큐를 자주 검사하지 않으면 부하 불균형 발생

따라서 적절한 검사 주기 설정이 중요하다.

[6] Linux 멀티프로세서 스케줄러

Linux 커뮤니티에서는 멀티프로세서 스케줄러를 위한 단일화된 방식이 존재하지 않았다. 대표적으로 다음과 같은 세 가지 스케줄러가 등장하였다.

- O(1) 스케줄러
- Completely Fair Scheduler(CFS)
- BFS 스케줄러(Brain Fuck Scheduler)

이들 스케줄러의 장점과 단점에 대한 자세한 내용은 Meehan 의 학위 논문 [Mee11]에서 확인할 수 있다. 여기서는 기본적인 아이디어만 간단히 요약한다.

O(1) 스케줄러와 CFS 는 멀티 큐 방식을 사용하고, BFS 는 단일 큐 방식을 사용한다. 이 사실은 단일 큐 방식과 멀티 큐 방식이 모두 실제 시스템에서 성공적으로 사용될 수 있음을 보여준다.

물론 이 스케줄러들을 구분하는 기준은 이것만이 아니다.

O(1)스케줄러

O(1) 스케줄러는 우선순위 기반(priority-based) 스케줄러이다. 이 방식은 이전에 살펴본 MLFQ 와 유사하다.

특징은 다음과 같다.

- 프로세스의 우선순위를 시간에 따라 동적으로 변경
- 가장 높은 우선순위를 가진 작업을 선택
- 특히 상호작용(interactive) 작업의 응답성을 중요하게 고려

Completely Fair Scheduler(CFS)

CFS 는 결정론적(deterministic) 비례 배분(proportional share) 방식을 사용한다. 이 방식은 이전에 설명했던 보폭 스케줄링(stride scheduling)과 유사한 개념이다. 즉, 각 프로세스가 공정하게 CPU 시간을 나누어 가지도록 설계된 스케줄러이다.

BFS 스케줄러

BFS 는 세 가지 스케줄러 중 유일하게 단일 큐 방식을 사용한다. 또한 비례 배분 방식을 기반으로 동작한다.

하지만 내부 알고리즘은 Earliest Eligible Virtual Deadline Frist(EEVDF)라는 더 복잡한 방식에 기반을 두고 있다.

[7] 요약

다양한 방식의 멀티프로세서 스케줄링을 살펴보았다.

단일 큐 방식(SQMS)은 구현이 용이하고 워크로드의 균형을 맞추기 용이하지만 많은 개수의 프로세서에 대한 확장성과 캐시 친화성이 좋지 못하다.

멀티 큐 방식(MQMS)은 확장성이 좋고 캐시 친화성을 잘 처리하지만 워크로드 불균형에 문제가 있고 구현이 복잡하다.

어떤 방식을 택하든지 쉬운 답은 없다.

CPU 스케줄러는 사소한 코드 수정으로 시스템의 동작이 엄청나게 바뀌기 때문에 모든 경우에 다 잘 동작하는 범용 스케줄러를 구현하는 것은 매우 어렵다.

본인이 하는 작업이 가져올 결과를 정확히 이해하고 있거나, 괜찮은 보수를 받는다면 한 번 시대에 불 만하다.

